

SoK: Reassessing Side-Channel Vulnerabilities and Countermeasures in PQC Implementations

Patrik Dobias¹, Azade Rezaeezade², Lejla Batina³, Łukasz Chmielewski⁴ and
Lukas Malina¹

¹ Brno University of Technology, Brno, Czech Republic, xdobia13@vut.cz, malina@vut.cz

² Delft University of Technology, Delft, The Netherlands, a.rezaeezade-1@tudelft.nl

3

Radboud University, Nijmegen, The Netherlands, lejla@cs.ru.nl

⁴ Masaryk University, Brno, Czech Republic, chmiel@fi.muni.cz

Abstract. Post-Quantum Cryptography (PQC) algorithms should remain secure even in the presence of quantum computers. Although the security of such schemes is guaranteed at the algorithmic level, real-world implementations often suffer from other vulnerabilities like Side-Channel Attacks (SCA). This Systematization of Knowledge (SoK) paper investigates side-channel attacks targeting implementations of PQC algorithms. This work categorizes attacks from an adversarial perspective to identify the most vulnerable components of the algorithms' implementations and highlights unexplored parts in current implementations. In addition, it reviews and analyzes the efficiency and efficacy of existing countermeasures to SCA in current hardware implementations. This approach helps identify countermeasures that provide broader protection and highlights characteristics needed for future secure implementations. Our findings offer guidance in strengthening existing systems and developing more efficient defenses against side-channel attacks.

Keywords: Side-channel Attacks, Post-Quantum Cryptography, Countermeasures

1 Introduction

Quantum computing poses a significant threat to classical public-key cryptographic systems. Shor's algorithm [Ber09] enables solving problems such as integer factorization and discrete logarithm, the foundations of widely used protocols like RSA and ECDH, in polynomial time on sufficiently powerful quantum computers. To maintain the security of cryptographic protocols in the presence of quantum adversaries, researchers have been developing new families of schemes based on mathematical problems believed to be resistant to quantum attacks. These so-called post-quantum schemes include lattice-based, code-based, hash-based, multivariate-based, and isogeny-based constructions. A pivotal moment in the transition to post-quantum schemes was the standardization process initiated by the National Institute of Standards and Technology (NIST) in 2016 [NIS17]. This effort led to the selection of two Key Encapsulation Mechanism (KEM) schemes: Kyber [BDK⁺18] and HQC [MAB⁺18], and three Digital Signature Algorithm (DSA) schemes: Dilithium [DKL⁺18], Falcon [FHK⁺18], and SPHINCS⁺ [BHK⁺19] for standardization. The NIST process continues, with additional signature schemes currently under evaluation [NIS22].

Although these schemes are based on hard mathematical problems, believed to be intractable even for quantum computers, theoretical security alone does not ensure their practical resilience. During the standardization process, NIST emphasized the importance of implementation-level security, particularly resilience against side-channel and

fault-injection attacks. This emphasis has driven extensive research efforts focused on both attacking and defending practical PQC implementations. Significant attack research has exposed various vulnerabilities; early examples demonstrated how timing variations in components such as error-correcting decoders could be exploited [STM⁺08, SSMS10], and this remains relevant in more recent attacks [WTBB⁺19, PT20]. Breakthroughs include the first single-trace side-channel attack on a lattice-based scheme [PPM17] and targeted attacks on specific algorithmic components, such as the floating-point arithmetic used in Falcon [KA21]. More broadly, attacks against the Fujisaki-Okamoto (FO) transform [UXT⁺21], an operation used in multiple KEM schemes, demonstrate that similar attack techniques can be effective even across diverse mathematical foundations. Key contributions among countermeasures against these attacks include the first masked implementation of an R-LWE-based scheme [RSRVV15], which applied the foundations for a wide range of countermeasures. Since then, efforts have focused on hardening specific components or entire phases to prevent side-channel leakage.

With so many new attacks emerging in recent years, each using different techniques and targeting various parts of PQC algorithm implementations, it has become increasingly complex to get a clear understanding of the current state of their vulnerabilities and the associated countermeasures. With this SoK, our objective is to give a broad overview and organize known attacks and countermeasures into a clear structure, making it easier for readers to understand the different attack scenarios and see how they might apply to other schemes or implementations. We also provide an open-source repository¹, organized using Obsidian², which contains a curated list and some high-level information on prior side-channel attacks on PQC implementations in the focus of this work. Our analysis focuses on the cryptographic schemes selected for NIST’s standardization and those that advanced to the fourth round [ACD⁺22]. We analyze side-channel attacks on both software and hardware implementations, as attacks targeting software are also often relevant to hardware implementations. Although these attacks may be more challenging to perform in hardware due to the unique characteristics of hardware systems, the underlying vulnerabilities often remain applicable. By examining both implementation types, we aim to provide a comprehensive understanding of the potential risks and challenges across different platforms.

This work can serve as a valuable resource for two primary audiences within the SCA community. First, it offers the academic and research community a structured and comprehensive overview of the current landscape of side-channel attacks on PQC schemes, more focused on those poised for deployment in quantum-resistant security protocols. By combining existing attack techniques and countermeasures, this work gives an overview of common vulnerabilities and points out areas needing more research for secure real-world use. Second, this study is intended to support practitioners involved in testing or certifying implementations, both in hardware and software, by offering a reference point and guide for their PQC implementation evaluations.

Related Work. In recent years, numerous studies have dealt with secure implementations of PQC schemes and their protection against various attacks. Most of these works focus on secure software implementations of PQC while also mentioning relevant attacks on hardware implementations. For instance, Howe *et al.* presented a SoK paper [HPA21] focusing on the design and implementation issues with PQC that includes a section dedicated to SCA and countermeasures. Xagawa *et al.* [XIU⁺21] studied NIST PQC Round three Key Encapsulation Mechanism (KEM) candidates [Nat21] from the perspective of fault-injection attacks. They consider only software implementations of quantum-resistant KEM schemes. In addition, Ravi *et al.* [RCDB24] thoroughly analyzed possible attacks and countermeasures for software implementations of Kyber and Dilithium. They focused

¹<https://github.com/Paprikadobi/Reassessing-SCA-Vulnerabilities-of-PQC-Implementations>

²Obsidian is a note-taking and knowledge management application that stores content in local Markdown files. <https://obsidian.md>

on both side-channel and fault-injection attacks. Their scenarios include single- and multi-trace attacks depending on the reuse of the secret keys, targeting key generation, encapsulation and decapsulation phases in Kyber, and key generation and signing phases in Dilithium. Targeted operations are NTT, Keccak, and message encoding/decoding. For every scenario, they offer possible countermeasures, such as shuffling coefficients during NTT and coefficients sampling from Keccak or masking by splitting secret parts into multiple random shares. Later, Shaller *et al.* [SZN23] provided a comprehensive review of attacks and countermeasures in PQC. They focused on all standard PQC families (code, lattice, isogeny, multivariate, and hash). Canto *et al.* [CKKA23] analyzed the state-of-the-art SCA and corresponding countermeasures for all schemes currently considered in NIST standardization. Aulbach *et al.* [ACK24] recently published a SoK paper on physical attacks against UOV-based signature schemes, which is highly relevant for the ongoing NIST standardization of additional signature schemes [NIS22]. Their work provides a comprehensive overview of known side-channel and fault-injection attacks on UOV-based schemes.

The only survey with a primary focus on SCA threats on hardware platforms or the secure hardware implementations of PQC was presented by Jedlicka *et al.* [JMS⁺22]. They analyzed state-of-the-art works on side-channel attacks on post-quantum schemes and available countermeasures. The analyzed works include attacks on error samplers, cold boot attacks, and specific attacks on McEliece, Rainbow, Falcon, Dilithium, and Kyber schemes. Furthermore, they proposed the hardware implementation of hiding in the time domain that should be used as a universal basic countermeasure, which should be included in any existing implementation. However, the authors provided only a limited review in the field of SCA and PQC.

Even though there are multiple studies in recent years, these studies differ from ours in two important aspects. First, these studies classify the SCA according to the specific PQC scheme attacked. This could be a limiting point of view as the attacks might use similar approaches and apply to different schemes. Second, our work provides a more comprehensive and detailed study with a wider range of attack techniques and focuses on specific operations in KEM and DSA schemes.

Contributions. In this work, we make the following contributions:

- **Comprehensive Survey and Analysis of SCA:** We conduct a thorough assessment of side-channel vulnerabilities in both hardware and software PQC implementations, covering a wide range of attack techniques. The attacks are analyzed in three main sections: Section 3 focuses on attacks against KEM schemes with static key setup, Section 4 addresses attacks in ephemeral key setups, and Section 5 explores side-channel vulnerabilities in DSA schemes.
- **Combination of SCA with Cryptanalysis:** We explore practices in which SCA can improve traditional cryptanalytic methods. Specifically, we investigate how the partial information recovered through SCA can be leveraged to break PQC schemes, offering a new dimension to the cryptanalysis of PQC (Section 6).
- **Evaluation of Countermeasure Costs:** We provide an in-depth evaluation of the effectiveness and costs associated with countermeasures against SCA when implemented on hardware and software, focusing on identifying practical and efficient solutions for secure implementations (Section 7).
- **Highlighting Overlooked Areas and Guideline Development:** We structure known SCA attacks based on their target operations and attack models, enabling the identification of commonly exploited vulnerabilities in PQC implementations (Section 8). This classification supports the development of practical evaluation flowcharts which can be used systematically for evaluating implementation and highlights underexplored attack vectors, offering strategic direction for future research.

2 Preliminaries

2.1 Type of Implementations

Implementations of cryptographic algorithms can be divided into three categories based on the target platform: software, hardware, and co-design.

In **Software Implementations**, instructions are typically pipe-lined and executed sequentially within a processor or, in some cases, in parallel across multiple cores. The pipelining and sequential processing leads to direct leakage of processed instructions and data. The Hamming weight model is particularly suitable for software leakage analysis because it reflects the number of '1' bits in the processed data, directly influencing power consumption [BCO04a]. The identity leakage model is another suitable leakage model for software implementation, which reflects the exact value of processed data. For some examples of software implementation of PQC, one can look into [KPR⁺] (multiple PQC schemes for ARM Cortex-M4) [HKL⁺22] (masked Kyber), [BDK⁺21] (masked Saber), [DKL⁺24] (reference implementation of Dilithium).

In **Hardware Implementations**, the processing often occurs in parallel across different components of a circuit. The leakage in hardware is mainly correlated with the transitions between different states of the circuit, as changes in bit values (from '0' to '1' or '1' to '0') consume more power. Therefore, the Hamming distance model, which measures the difference between two binary states, is commonly used in hardware leakage analysis [BCO04a]. Many works present hardware implementations of PQC, such as [GJCJ23, SAW⁺23, LVC24] that implement DSA schemes and [XL21, RBMG21, CCD⁺22, DXN⁺23] that implement KEM schemes.

In **Co-design Implementations**, a hardware implementation is coupled with a software implementation. As an example, Banerjee *et al.* [BUC19] coupled an ASIC implementation of the arithmetic and sampling modules required for lattice-based key exchange and signature algorithms with an efficient RISC-V microprocessor to improve energy and area efficiency. This kind of implementation gives us some of the advantages of hardware and software implementation. To attack this kind of implementation, a more complex analysis is needed.

2.2 PQC Schemes

Key Encapsulation Mechanism is an asymmetric cryptographic algorithm (using a pair of public and private keys) that allows two (or more) parties to securely share a symmetric cryptography key (referenced as the session key in this work) over an insecure communication channel. A KEM does this exchange using three phases: key generation, key encapsulation, and key decapsulation. Generic algorithms for these phases are described in Algorithm 1, with secret information that can be leaked highlighted in red color. The key generation and key encapsulation phases depend on a random value generated during the phase. Thus, they are vulnerable only to single-trace attacks. The key decapsulation phase is a more interesting target for attackers, as it uses secret key to decrypt ciphertext and depending on the setup (static or ephemeral) can be used even in attacks requiring multiple traces. Moreover, the ciphertext that is being decrypted can be chosen by the attacker, giving him even more capabilities.

Digital Signature Algorithm is an asymmetric cryptographic algorithm that is used to create and verify digital signatures, ensuring the authenticity and integrity of digital messages. A DSA comprises three phases: key generation, signing, and verification. These phases are shown in Algorithm 2, with secret information that can be leaked highlighted in red color. From these, we can see that only the key generation and signing phases can be attacked to retrieve some secret information.

- **KeyGen()**: samples secret key and computes corresponding public key. Returns (pk, sk) .
- **Encrypt** (pk, m) : encrypts message using public key. Returns c .
- **Decrypt** (sk, c) : decrypts ciphertext using secret key. Returns m .
- **Encapsulate** (pk) : generates random message $m \leftarrow \mathbb{B}$, computes corresponding ciphertext $c \leftarrow \text{Encrypt}(pk, m)$, and derives shared key using message and some public values $K \leftarrow \text{KeyDerive}(m, [pk, c])$. Returns (K, c) .
- **Decapsulate** (sk, c) : decrypts message $m' \leftarrow \text{Decrypt}(sk, c)$, re-encrypts the decrypted message $c' \leftarrow \text{Encrypt}(pk, m')$, **if** $c = c'$ **then** derives shared secret as $K \leftarrow \text{KeyDerive}(m, [pk, c])$ **else** derives shared secret as $K \leftarrow \text{KeyDerive}(sk, [pk, c])$. Returns K .

Figure 1: Key Encapsulation Mechanism phases with potential leakage points marked in red.

- **KeyGen()**: samples secret key and computes corresponding public key. Returns (pk, sk) .
- **Sign** (sk, m) : signs hashed message using secret key. Returns σ .
- **Verify** (pk, m, σ) : verifies signature. Returns true | false.

Figure 2: Digital Signature Algorithm phases with potential leakage points marked in red.

2.3 Attack Techniques

Cryptographic algorithms are designed to be secure against mathematical attacks. However, when implemented and run on a physical device, side-channel leakages such as power consumption, electromagnetic emanation³, or timing are highly relevant to the processed data. An attacker can use these leakages to evaluate hypotheses about the processed data. If the link between the processed data and the secret data (for example, the key or the message) is clear, then the attacker can deduce the secret data [KJJ99]. Countermeasures, such as masking and hiding, are used to weaken the connection between secret data and information from the side channel. In this section, we briefly look into the widely known attack techniques. There are other kinds of SCA, like cache attacks [OST06] or other tools that are used for SCA, like clustering [MBBB16], which are not mentioned in this work because there have been fewer studies on PQC implementations.

2.3.1 Timing Attacks

In timing attacks [Koc96], the attacker exploits the non-constant time execution of operations within a cryptographic algorithm to recover the secret value. These are some of the simplest attacks to prevent⁴, but can still occur.

³In most of the cases attacks, which are possible with power analysis, are possible with electromagnetic emanation and vice versa.

⁴Many implementations consider constant-time implementation, eliminating execution-time-related leakages to an acceptable extent.

2.3.2 Non-profiled Attacks

Non-profiled attacks are mainly attacks that do not rely on building profiles to characterize the leakage of sensitive data at the attack point. Instead, these attacks utilize only the traces from the device under attack (the target device) along with statistical tools to extract the secret. The non-profiled category includes simple, differential, and non-profiled deep learning-based attacks.

Simple Power Analysis (SPA)⁵ involves attacks where measured leakage is directly interpreted to get information about executed operations [KJJ99]. For this analysis to be effective, there must be a strong dependency between the leakage and the secret data.

Differential Power Analysis (DPA) can be used when it is not possible to interpret the traces directly. This analysis uses statistics on multiple measurements to extract secret information from them [KJJ99].

Correlation Power Analysis (CPA) is an extension of DPA that relies on correlation analysis to identify the relationships between the leakage and the secret data [BCO04b].

Clustering-based Attack is using clustering techniques to put similar traces (or similar segments of a single trace) in the same buckets and decide about the processed data based on each cluster [WHW⁺25].

2.3.3 Profiled Attacks

In profiled attacks, the attacker has full control over a clone of the device under attack and can profile the characteristics of the leakage using the clone [CRR03]. From an information theory perspective, these kinds of attacks are a strong [SKS09]. Profiled attacks require profiling and attack stages. During the profiling stage, the attacker gathers enough measurements from the clone device and builds the profiles. During the attack stage, the attacker matches the traces from the device under attack with the built profiles. There are different types of profile attacks, and among them, template attacks [CRR03, CK13] and deep learning-based attacks [LBM15, HZ12] are widely known. Profiled attacks enable single-trace attacks, which are also effective in ephemeral key setups.

Template Attack (TA) involves templates, which are random variables with certain probability density functions (mostly Gaussian). To specify templates, the attacker must specify the distribution parameters (mean and variance or covariance in the Gaussian case) using the experimental data. The attack uses estimation theory and the maximum likelihood method to estimate the required parameters. This type of attack faces two main challenges: collecting many traces from the clone device to build templates and assuming an underlying distribution (e.g., Gaussian), which may not match the actual data distribution.

Deep Learning-based Attacks (DL-SCA) consist of training and testing stages, which completely comply with the concept of profiling and attack stages. In DL-SCA [MPP16], the templates are not built separately. Instead, a deep neural network is used to classify the traces based on the values of the sensitive variable targeted. Commonly used neural network architectures for classification include multilayer perceptron (MLP) and convolutional neural network (CNN).

2.3.4 Soft Analytical SCA (SASCA)

These attacks start by performing a profiling attack (TA, DL-SCA, or any other technique) to gather vectors of probabilities for multiple sensitive variables, including the key. Then, these probability vectors are processed further using a factor graph that models the

⁵In some references, when the measurements in use are electromagnetic emanations, the terms Simple Electromagnetic Analysis (SEMA), Differential Electromagnetic Analysis (DEMA), and Correlation Electromagnetic Analysis (CEMA) are used. For simplicity, we do not differentiate between power and electromagnetic measurements in this SoK.

algorithm and its implementation, incorporating the collected probabilities into the graph. The probabilistic inference algorithm computes marginal probabilities for all inputs, outputs, and intermediates, including key components using belief propagation [VCGS14]. This kind of attack is based on profiling. However, the additional step of inference using the probabilities can distinguish it from the plain profiled attacks.

Note: The definitions provided here are based on their reference introductions, which are very general. In practice, the attacks sometimes deviate from the general definitions while they are still strong enough to retrieve the secret data. For instance, Ravi *et al.* [RBRC22] use templates to profile the leakage of the message’s Hamming weight without any access to a clone device and key information, which is usually used in a standard template attack. We clarify the deviation when the change is important from an adversarial model perspective (see Section 2.4).

2.4 Adversarial Models

Adversarial models define the attacker’s capabilities, which we subdivide into the following three primary types in this SoK.

Non-interactive Adversarial model represents the most constrained adversary, who can only observe the cryptographic system’s execution and collect side-channel leakages. Throughout this work, we refer to this adversarial model using the letters “NI” in the tables.

Interactive Adversarial model represents a more capable adversary who can measure side-channel leakages (as in the non-interactive model) and actively interact with the cryptographic system by providing inputs (e.g., chosen ciphertexts). Throughout this work, we refer to this adversarial model using the letter “I” in the tables.

Adversary with Access to a Clone model represents an adversary, non-interactive or interactive, who additionally has prior access to a clone device. The adversarial freedom varies across different works. For example, in some works, the adversary disables countermeasures, while in others, they assume complete control over the key and plaintext/ciphertext. Using this clone, the adversary can perform profiling to enhance their attack strategies. Throughout this work, we refer to this adversarial model using the letter “D” in the tables.

2.5 Our Methodology for Categorizing PQC-SCA

In this work, side-channel attacks on KEM are divided into attacks in **static** and **ephemeral** key setup, based on keypair refresh frequency. In a static setup, the same key pair is reused for multiple key exchanges, while in an ephemeral key setup, a new key pair is used for every key exchange. In most cases, these setups directly correspond to the number of traces available for the attack. In a static key setup, multiple-trace attacks are possible. In contrast, in the ephemeral key setup, only single-trace attacks are possible⁶. While this division theoretically applies to DSA schemes, our research did not find a significant focus on it for them. The SCA on KEM in static and ephemeral key setups have been studied in Section 3 and Section 4, respectively. The SCA on DSA has been studied in Section 5.

The PQC algorithms considered in this work include the KEM and DSA schemes that were selected for the NIST’s standardization and those that advanced to the fourth round [ACD⁺22]. It is worth mentioning that the final report of the fourth round of the NIST competition was published in March 2025, and the HQC will be standardized based

⁶Attacks that are possible with a single trace are not always applicable in the ephemeral key setup. For example, the secret key recovery attacks assisted with chosen-ciphertext cannot be applied in an ephemeral key setup as the decryption of the chosen ciphertext revokes the current ephemeral keypair.

on the report [AAC⁺23]. Table 1 shows the list of the schemes (both KEM and DSA) considered.

Table 1: The list of schemes we consider in this work.

	KEM Schemes	DSA
Schemes selected to be standardize	Kyber (ML-KEM) HQC	Dilithium (ML-DSA), Falcon, SPHINCS ⁺
Remaining fourth round candidates	BIKE, McEliece	

The targeted schemes can also be categorized based on the specific mathematical problems they rely on for security. Five mathematical hard problems were used to design the PQC algorithms: lattice-based problems, such as the Shortest Vector Problem (SVP) and Learning With Errors (LWE), which involve finding approximate solutions in high-dimensional lattices; code-based problems, like decoding random linear codes; hash-based cryptography, which utilizes the preimage and collision resistance of cryptographic hash functions; isogeny-based problems, which focus on the difficulty of finding isogenies (mappings) between elliptic curves over finite fields, and multivariate polynomial problems, which rely on the difficulty of solving systems of nonlinear equations. These mathematical problems are computationally intensive for classical computers and remain intractable for quantum computers, making them ideal for ensuring long-term security in a quantum computing era. In the side-channel analysis, the focus so far has been mainly on lattice-based and code-based schemes, which were the most successful during the NIST standardization process. We also include an attack on the hash-based scheme SPHINCS⁺. However, there are not many side-channel attacks against this scheme, but rather fault-injection attacks, which we do not consider in this SoK.

Figure 3 illustrates the hierarchical structure discussed in this section, emphasizing an additional level of detail: the specific operations targeted by SCA so far. Classifying SCA works by their target operations offers two main advantages. First, it provides an overall view of vulnerabilities within each family. For example, pointwise multiplication, a fundamental operation in lattice-based PQC algorithms (e.g., Kyber as KEM, and Dilithium and FALCON as DSA), can be a shared vulnerability across all schemes in that family. Besides, an operation like the Fujisaki-Okamoto (FO) transform, targeted in both lattice-based and code-based schemes, highlights potential cross-family vulnerabilities. Second, this classification guides the development of countermeasures by highlighting shared properties within and between families, allowing for more generic and effective solutions.

3 Side-channel Attacks on KEM Schemes in Static Key Setup

This section focuses only on attacks mounted in a static key setup. As these attacks require multiple traces, one common countermeasure would be to use each key only once, thereby switching to an ephemeral key setup. One can see Figure 3 for a top-level overview.

3.1 Attacks on Lattice-based Schemes

This section presents existing attacks against lattice-based KEM schemes in a static key setup categorized according to the targeted operation. A summary of attacks is provided in Table 2. The table provides a summary of the attacks, including the targeted

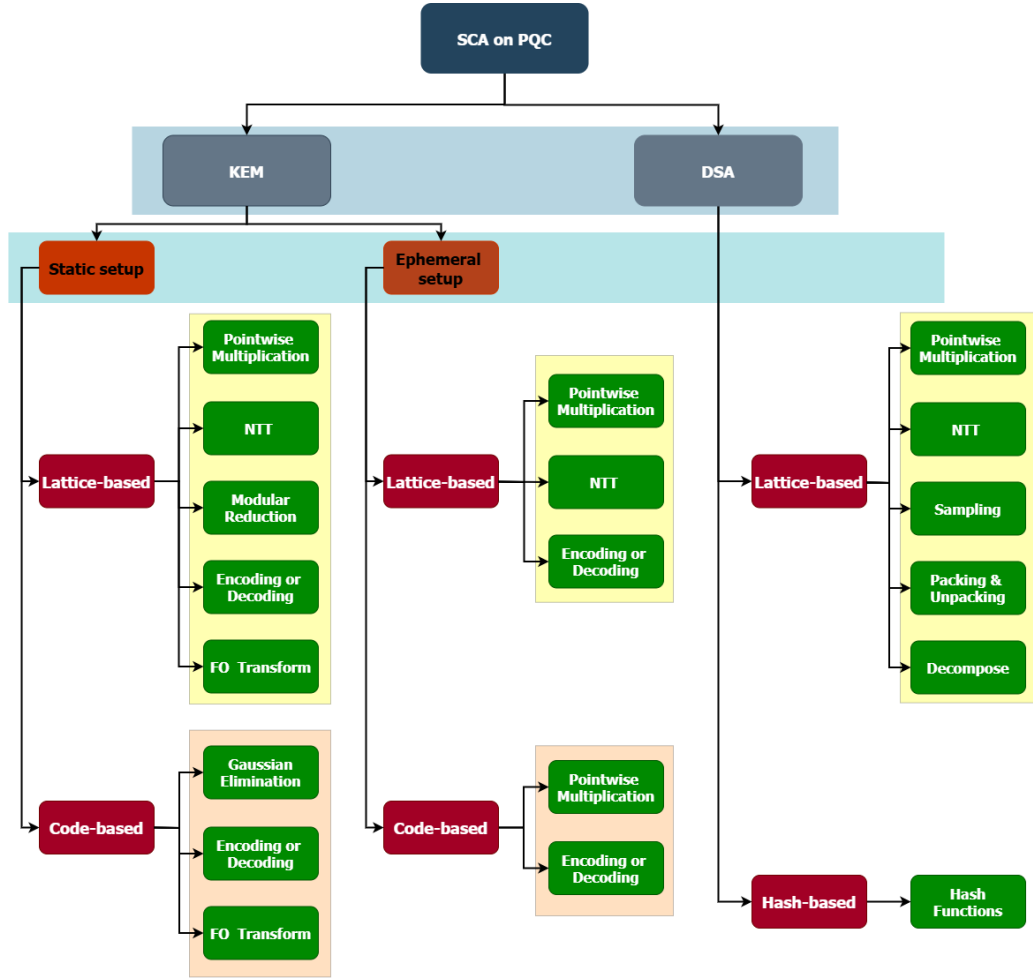


Figure 3: Categorization of side-channel attacks according to their application, underlying mathematical problem and operation that has been target.

operation, references to works detailing the attacks, information about the attacked implementation (hardware, software, and applied protections), techniques used to obtain leakage information, the adversary’s capabilities, and possible countermeasures. The countermeasures listed are primarily those proposed by the authors of the respective attacks. Countermeasures applicable to all attacks in a static key setup (using an ephemeral key setup) or to most chosen ciphertext attacks (detecting malicious ciphertexts) are not specifically included. For articles that do not propose countermeasures, we have suggested possible solutions based on related attacks and common practices, which are presented in *italic*. The structure and description of the information provided in this table apply consistently across all tables summarizing attacks in subsequent sections.

3.1.1 Leakage during pointwise multiplication

Polynomial pointwise multiplication involving secret polynomials or message polynomials (or both) occurs in every phase of lattice-based KEM schemes, making it highly susceptible to attacks. Leakage during this operation or even during subsequent operations can lead to the recovery of the secret polynomial. The leakage can be further magnified in the interactive adversarial model by crafting malicious polynomials, such as chosen ciphertext

during decapsulation, which are then multiplied with the secret polynomials. In a static key setup, this operation is mainly targeted using **CPA** in both software [KdG21, MWK⁺24] and hardware [RBVB23, ZPM⁺23] implementations. In addition, all attacks are reported on the **unprotected** implementations.

Both attacks on software implementations used the Hamming weight model. Karlov and Linard de Guertechin [KdG21] required a minimum of 80 traces targeting Kyber at the lowest security level. Mujdei *et al.* [MWK⁺24] recovered the secret key attacking the pointwise multiplication of the fixed secret key and messages in NTT domain targeting Kyber⁷ using 200 traces. They needed to guess two coefficients at once and select the pairs with the largest correlation coefficient.

Both attacks on hardware implementations employ the Hamming distance model, which requires knowledge of the reference value. Rodriguez *et al.* [RBVB23] exploited the fact that in their implementation, the same unit performs both NTT and pointwise multiplication. During the decapsulation phase, before the ciphertext is multiplied with the secret key, it is first transformed using NTT, thus providing the reference values for the first multiplication. This allows for a simple countermeasure by performing random "dummy" multiplication between the NTT and pointwise multiplication, effectively obscuring the reference values. For all subsequent multiplications, the reference values depend on the success of retrieving the previous values. For a successful key recovery, around 160 thousand traces were required. Zhao *et al.* [ZPM⁺23] performed their correlation attack with specially crafted ciphertexts, requiring only about 1-2 thousand traces.

3.1.2 Leakage during NTT

Many lattice-based schemes perform polynomial multiplication in the NTT domain to improve performance. Conversion to and from this domain can operate on secret values, introducing another point of potential leakage. Xu *et al.* [XPR⁺22] exploited such a leakage in the reference implementation of Kyber [Tea] during the final step of INTT applied to the product result of ciphertext and secret key, where this product result is multiplied by a constant value. This multiplication directly reveals the Hamming weight of the product result, which can be compromised with an **SPA**. They further demonstrated that it is possible to uniquely distinguish each coefficient of the secret key polynomial by using maliciously crafted ciphertexts, requiring as few as two traces per polynomial. The implementation used is **unprotected**.

Another leakage during NTT was explored by Hamburg *et al.* [HHP⁺21], who used SASCA in addition to chosen ciphertexts to provide additional information on coefficients that help during belief propagation. Their method involved constructing sparse ciphertexts with a controlled number of non-zero coefficients and evaluating the conditions under which successful secret key recovery could be achieved. A key finding of their study was the feasibility of a k-trace attack (with $k = 2, 3$, or 4 , depending on Kyber's security level) using 64 non-zero coefficients. Additionally, by increasing the number of non-zero coefficients and reducing noise, they demonstrated the practicality of executing single-trace attacks.

3.1.3 Leakage during Modular reduction

Modular reductions like Barrett and Montgomery reductions are used for efficient arithmetic operations in finite fields or rings. These reductions can replace expensive division operations with less expensive multiplication and shift operations. Sim *et al.* [SPH22] proposed a chosen ciphertext attack on the reference implementation of Kyber [Tea] by focusing on Barrett reduction used during the decapsulation. Using the Hamming

⁷They did a comprehensive CPA on Toom-Cook and NTT multiplication for various lattice-based algorithms including Saber and NTRU, but those schemes are out of scope for this work.

weight leakage from this operation and chosen ciphertext, which could magnify the deference of single secret key coefficients, Sim *et al.* [SPH22] recover the secret key in the normal domain (values in $\{-2, -1, 0, 1, 2\}$) using an **SPA** and simple clustering techniques. The implementation in use is **unprotected**. With their attack, secret key recovery is possible with 6, 9, and 12 traces for Kyber512, Kyber768, and Kyber1024, respectively. The countermeasures proposed to limit this attack are masking and shuffling. Zhao *et al.* [ZPM⁺23] used the same vulnerability to attack an unprotected hardware implementation of Kyber using CPA.

3.1.4 Leakage during message encoding and decoding

Message encoding and decoding directly manipulate the coefficients of the secret message, which serve either as the session key or as the basis for deriving it. Leakage during these operations can compromise the security of the session. Additionally, the attacks analyzed demonstrate techniques for recovering the long-term secret key through chosen ciphertexts. Notably, with the exception of [XPR⁺22], all attacks described in this section rely on multiple traces to recover the message. Although this approach is impractical for session key recovery, as each message is typically decrypted only once, it remains effective for recovering the long-term secret key.

Xu *et al.* [XPR⁺22] used a simple electromagnetic analysis to retrieve the message coefficients during decoding. Their attack leverages leakage arising from significant differences in the Hamming weight of the mask value used to decode each coefficient to a message bit. The attack was reported for an **unprotected** implementation of Kyber in [KPR⁺]. This is a single work using **SPA** to attack decoding. As one can see, the rest of the works use deep learning to attack both unprotected and protected implementations.

Bernstein *et al.* [BBB⁺25] exploited a variable-time division instruction in the reference implementation of Kyber. Although divisions by constants are typically replaced with constant-time multiplications by precomputed constants, they identified scenarios that enabled compiler flags for code size optimization, where this replacement does not occur. As a result, they were able to exploit the message decoding process, which involves division modulo Kyber’s modulus q . They discovered that certain inputs fall into one of three distinct value ranges, each with a different execution time for division. By crafting ciphertexts accordingly, they successfully recovered the secret key within a few hours. Moreover, they found that this type of variable-time behavior affects over a thousand cryptographic implementations included in the SUPERCOP benchmarking suite.

DL-SCA against unprotected implementations. Wang *et al.* [WND23] trained MLP models to classify message bytes in software implementations. They explored how bias, typical of neural networks, can be used to increase the classification success rate with their approach, called multi-bit error injection. In this approach, for a ciphertext representing some message, 255 other ciphertexts are generated in such a way that each byte of the message has all possible values, and the traces corresponding to these ciphertexts are captured. These traces are then fed to the model, and the most likely byte value is determined based on overall probabilities. Ji *et al.* [JWN⁺23] used MLP models to classify message bytes. However, Ji *et al.* modified the approach for a hardware implementation [XL21], where multiple bytes are processed in parallel, by introducing sliced multi-bit error injection. The main difference is that the error is applied only to every n -th byte of the message (in their work, every fourth), corresponding to the number of parallel processed bytes.

Wang *et al.* [WCCL24] explored the practicality of leakage in hardware implementations, arguing that the works of [RBVB23, JWN⁺23] are impractical. Their analysis of multiple implementations identified two primary challenges: first, targeting a single bit is difficult because multiple bits are processed in parallel, leading to significant noise from changes in other bits; second, the Hamming distance model requires knowledge of a reference value. To

address these issues, they proposed a novel attack on message encoding leveraging chosen ciphertexts. Their approach ensures that only a single bit in the parallel processing changes, effectively eliminating noise from other bit transitions and guaranteeing known reference values. They experimentally verified their attack against the hardware implementation of Kyber, first trying simple analysis, which was, however, not applicable, as it was difficult to distinguish between bits, so they opted to use a deep learning model. The authors further observed that their attack remained effective against protected implementations using masking and shuffling countermeasures.

DL-SCA against protected implementations. Wang *et al.* [WCCL22] and Backlund *et al.* [BNGD23] demonstrated that even masking and shuffling protected implementations are vulnerable. They used a deep learning-based approach to recover the message during the encoding step of re-encryption in the key decapsulation phase. Wang *et al.* [WCCL22] showed that a neural network is capable of classifying the message bytes based on sub-traces of encoding that correlate to message shares in masked software implementation [HKL⁺22]. Backlund *et al.* [BNGD23] used two neural networks, as they target masked and shuffled software implementation [HKL⁺22, BDK⁺21], where the first network recovers Fisher-Yates indices used during shuffling, and the second network extracts the corresponding bits. In their attack, they were able to recover only the 0th and 255th indices in a single trace due to their large Hamming weight difference. To recover all bits, they iteratively updated the ciphertexts. Backlund *et al.* [BNGD23] work is an extension of Ngo *et al.* work in [NDGJ21] (masked protected implementation) and [NDJ21] (masked and shuffled protected implementation), where they used a similar technique to recover the session key and the secret key from the decoding operation in a Saber.

Dubrova *et al.* [DNGW23] focused on developing an efficient method to attack higher-order masking schemes. They presented a novel training technique, recursive learning, which enables neural networks to be trained for higher-order masking by initializing weights from networks trained on lower-order maskings, thus reducing training time significantly. This approach successfully recovered the message bit-by-bit from a fifth-order masked software implementation of Kyber, and the authors suggest that their method could be applicable to other lattice-based KEM schemes.

3.1.5 Leakage during FO transform

FO transform [FO99] ensures that a decrypted message is outputted only for valid ciphertexts, thus thwarting chosen ciphertext attacks (CCA). However, side-channel leakage, exposing information about the decrypted message during this transform, can degrade the indistinguishability under chosen ciphertext attack (IND-CCA) security of the scheme to indistinguishability under chosen plaintext attack (IND-CPA). As this transform consists of re-encryption of the decrypted message and comparison of the ciphertexts, it offers multiple operations that can be targeted. Because of this, many works propose attacks without relying on specific leakage points, exploiting the additional information on decrypted messages to construct oracles, which are then queried with chosen ciphertexts to recover the secret key. In the following, the attacks exploiting FO transform are divided, considering the oracle that they use, which is more relevant considering this attack point.

Decryption-failure oracle is capable of indicating whether the decryption of two similar ciphertexts results in the same decrypted message, gathering information about the secret key that can lead to full key recovery. Guo *et al.* [GJN20] constructed this oracle by targeting the non-constant-time implementation of ciphertext comparison at the end of the decapsulation phase. They crafted similar ciphertexts so that those decrypting to the same message would have identical leading bytes, thus creating a timing differential: when the messages are the same, the first bytes match and the comparison takes longer until a later discrepancy is identified; when the messages differ, the first bytes are likely different, resulting in immediate rejection. Another attack using this oracle was

presented by Bhasin *et al.* [BDH⁺21], who used power consumption leakage during the ciphertext comparison, specifically targeting two masked comparison methods proposed in [OSPG16, BPO⁺20]. They identified vulnerabilities in the proposals and experimentally validated that the implementation would still leak when integrating both countermeasures into the Kyber implementation available in the pqm4 library. To distinguish differences in power consumption, they employed Welch’s t-test. Both countermeasures exhibited leakage, particularly in cases of borderline decryption failures. Furthermore, they demonstrated a collision attack on the proposal of Bache *et al.* and proposed corrections for both countermeasures.

Plaintext-checking oracle is used to distinguish between two values of the decrypted message when the ciphertexts are specially crafted so that the decrypted message can have only one of the two values. Ravi *et al.* [RRCB20] used electromagnetic leakage to instantiate such an oracle. To classify measured traces into one of two classes, they adopted Welch’s t-test-based reduced template approach. They demonstrated the generality of their attack by targeting multiple lattice-based schemes, including Kyber, Saber, and Frodo. Ueno *et al.* [UXT⁺21] provided a more general attack, which is also applicable to code-based and isogeny-based schemes. They particularly focused on the leakage of pseudorandom functions and pseudorandom number generators. To construct the plaintext-checking oracle, they classify the measured traces using a convolutional neural network. Bernstein *et al.* [BBB⁺25] also leveraged their timing attack on division, described in the previous subsection, to construct a plaintext-checking oracle. This was possible because division is also used during ciphertext compression as part of the FO transform. This attack is even more powerful than the one targeting message decoding, enabling secret key recovery within a few minutes. In particular, they showed that the attack is effective even with a remote adversary.

The main issue with attacks based on plaintext-checking oracles is that they only retrieve one bit of information about the secret key per trace. The question of increasing the amount of information and thus reducing the required number of traces was explored by Rajendrand *et al.* [RRD⁺23] and Tanaka *et al.* [TUX⁺23]. These works improved the attacks [RRCB20] and [UXT⁺21], respectively. Rajendrand *et al.* showed how to construct ciphertexts such that multiple bits of the message can be targeted in parallel, sticking to Welch’s t-test-based reduced template approach. Tanaka *et al.* extended the convolutional network to enable the classification of multiple classes.

3.2 Attacks on Code-based Schemes

This section presents existing attacks against code-based KEM schemes in static key setups categorized according to the targeted operation. A summary of attacks is provided in Table 3, with the same structure as described in Section 3.1.

3.2.1 Leakage during Gaussian elimination

Brinkmann *et al.* [BCM⁺23] attacked the key generation step of McEliece in the Classic and Botan implementations. Both implementations are **unprotected** software implementations. Many implementations store the initial random seed instead of bulky secret and public keys and reconstruct them on request. One of the main steps of public key reconstruction is performing Gaussian elimination on a binary matrix H . The power consumption (or electromagnetic emission) observed during this elimination correlates with the Hamming weight of the resulting eliminated binary matrix entries, which is sufficient to recover the secret. Since the attack exploits the key generation from the stored random seed, an ephemeral key setup is the best way to eliminate it.

Table 2: Summary of side-channel attacks on lattice-based KEM schemes in static key setup.

Target	Ref.	HW/SW	Technique	Adversarial	Possible Countermeasures
Pointwise Multiplication	[KdG21]	SW	CPA	NI	<i>masking</i>
	[RBVB23]	HW	CPA	NI	dummy operations
	[ZPM ⁺ 23]	HW	CPA	I	masking
	[MWK ⁺ 24]	SW	CPA	NI	masking, hiding
NTT/INTT	[XPR ⁺ 22]	SW	SPA	I	shuffling
	[HHP ⁺ 21]	SW*	SASCA	I + D	shuffling
	[ZPM ⁺ 23]	HW	CPA	I	masking
Barrett Reduction	[SPH22]	SW	Clustering	I	masking, shuffling
	[ZPM ⁺ 23]	HW	CPA	I	masking
Message Encode/Decode	[XPR ⁺ 22]	SW	SPA	I	shuffling
	[BBB ⁺ 25]	SW	timing	I	-
	[WCCL22]	SW†	DL-SCA	I	-
	[BNGD23]	SW‡	DL-SCA	I + D	-
	[WND23]	SW	DL-SCA	I	-
	[JWN ⁺ 23]	HW	DL-SCA	I	-
	[DNGW23]	SW†	DL-SCA	I + D	-
	[WCCL24]	HW	DL-SCA	I	masking
FO Transform	[GJN20]	SW	timing	I	constant-time
	[RRCB20]	SW	DPA	I	masking
	[BDH ⁺ 21]	SW†	DPA	I	comparison modification
	[UXT ⁺ 21]	SW	DL-SCA	I	masking, threshold implementation
	[TUX ⁺ 23]	SW	DL-SCA	I	-
	[RRD ⁺ 23]	SW‡	DPA	I	masking
	[BBB ⁺ 25]	SW	timing	I	-

HW/SW: † - masked, ‡ - shuffled, * - simulated

3.2.2 Leakage during message encoding and decoding

Message encoding and decoding are based on the selected error correction code in code-based schemes. These operations are non-constant time in nature, but some implementations tried to fix this. We classify the attacks regarding this consideration in the implementations as it is more relevant from the SCA perspective. A third thread of works uses this vulnerability as an oracle.

Non-constant time decoder. Constant-time implementations are essential for security, yet error-correcting decoders are often implemented such that their execution time varies depending on the number of corrected errors. This was exploited already in 2008 by Strenzke *et al.* [STM⁺08] and later by Shoufan *et al.* [SSMS10], who attacked the decoder in McEliece software implementations. Both attacks exploited the non-constant time execution by submitting modified ciphertexts to reveal the error bits corresponding to the encrypted message. More recent timing attacks were presented by Wafo *et al.* [WTBB⁺19] and Paiva and Terada [PT20], who presented timing attacks on the BCH decoder in HQC. Both attacks use a correlation between the weight of the decoded error and the execution time to recover secret keys. Wafo *et al.* also proposed modifications to the decoding algorithm to ensure constant-time execution.

Constant-time decoder. Even with constant-time decoders, similar attacks can be used by exploiting other types of leakage. Lahr *et al.* [LNPS20] used electromagnetic emanation to attack the software implementation of McEliece. They built on the timing attack developed by Shoufan *et al.*, adapting it into a differential attack. They further enhanced the attack by introducing a new method called "iterative chunking", which significantly reduces the number of required traces. Wang *et al.* [WPW⁺23] extended this

line of research by showing that such leakage could be exploited remotely through the Hertzbleed attack [WPH⁺22]. They showed that variations in the number of corrected errors cause changes in CPU frequency, which can be detected even by a remote client submitting decapsulation requests to a server. Guo *et al.* [GJJ22] proposed an attack to recover the secret key of Classic McEliece in both software and hardware implementations. Their attack exploits a general vulnerability in the decoding step, which can be exploited by decoding ciphertexts with a single error. By submitting ciphertexts with single errors at different positions, the secret support (a part of the secret key) can be progressively revealed. After a few hundred such submissions, the entire secret Goppa polynomial can be recovered. Guo *et al.* used deep learning-based classification to profile the values using power traces.

Oracle-based attacks. Goy *et al.* [GLG22] targeted the Reed-Muller decoder using electromagnetic emanation to construct an oracle capable of recognizing the number of corrected errors. For the assessment of classes, they used Welch’s t-test. Based on this oracle and using malicious ciphertext, they were able to recover part of the secret key required for successful decapsulation of session keys. Schamberger *et al.* [SHR⁺22] proposed a novel attack targeting the Reed-Solomon decoder. Using power leakage during the decoding process, they constructed a close-to-zero oracle capable of distinguishing whether the decoder output was an all-zero or non-zero codeword. To construct this oracle, they employed a template attack on the decoding function and a CNN attack on subsequent hash functions.

3.2.3 Leakage during FO transform

Similar to lattice-based schemes, multiple code-based schemes also rely on the FO transform to achieve IND-CCA security. Consequently, they are vulnerable to similar attacks. For example, the attack presented by Ueno *et al.* [UXT⁺21] was successfully demonstrated to recover the secret key for HQC and the partial key for BIKE.

Plaintext-checking oracle. Code-based specific attack on this transform using plaintext-checking oracle was presented by Guo *et al.* [GHJ⁺22]. This attack exploited non-constant time rejection sampling during the re-encryption of the message in HQC and BIKE schemes. Because rejection sampling uses seed derived from the message, they were able to construct a plaintext-checking oracle, which can distinguish whether two ciphertexts decrypt into two messages that yield different execution times. Using this oracle, with specially crafted ciphertexts, they were able to recover the secret key for both HQC and BIKE. Subsequent research [GNNJ23, SGG24, DG24] refined and improved this attack, reducing the required number of oracle calls. Notably, Dong and Guo [DG24] achieved the attack with the lowest number of oracle calls.

Table 3: Summary of side-channel attacks on code-based KEM schemes in static key setup.

Target	Ref.	HW/SW	Technique	Adversarial	Possible Countermeasures
Gaussian Elimination	[BCM ⁺ 23]	SW	CPA	NI	masking
Message Encode/Decode	[WTBB ⁺ 19]	SW	timing	I	constant-time
	[PT20]	SW	timing	I	constant-time
	[LNPS20]	HW	DPA	I	masking
	[GJJ22]	HW, SW	DL-SCA	I + D	-
	[GLG22]	SW	DPA	I	masking
	[SHR ⁺ 22]	SW	TA/DL-SCA	I	-
FO Transform	[UXT ⁺ 21]	SW	DL-SCA	I	masking or threshold implementation
	[GHJ ⁺ 22]	SW	timing	I	constant-time
	[GNNJ23]	SW	timing	I	constant-time
	[SGG24]	SW	timing	I	constant-time
	[DG24]	SW	DL-SCA	I	masking, shuffling

4 Side-channel Attacks on KEM schemes in Ephemeral Key Setup

In an ephemeral key setup, a new pair of secret and public keys is used for every key exchange, increasing the difficulty of the attack as it can be performed only using a single trace. One can see Figure 3 for a top-level overview of the families covered and the targeted operations in this setup.

4.1 Attacks on Lattice-based Schemes

This section presents existing attacks against lattice-based KEM schemes in ephemeral key setups categorized according to the targeted operation. All attacks are summarized in Table 4, following the same structure as described in Section 3.1. The only difference is that, for the ephemeral key setup, we indicate whether the secret key (SK) or the message (M), representing the session key, was leaked.

4.1.1 Leakage during pointwise multiplication

In an ephemeral key setup, the fact that the same secret coefficients are used multiple times with possibly different values, such as during matrix-vector or during Toom-Cook and Karatsuba multiplications, can be exploited to identify multiple points of interest (PoIs) for any coefficient in a single trace.

Matrix-vector multiplication. Yang *et al.* [YRZ⁺23] exploited this increased number of PoIs to introduce a single-trace attack that targets matrix-vector multiplication in Kyber. They showed that during the matrix-vector multiplication of Kyber, a single coefficient of the ephemeral secret key is multiplied by multiple coefficients of the public matrix. This kind of multiplication gives enough sub-traces from one key generation or key encapsulation operation, which can be used for a horizontal attack. To recover the Hamming weights of the coefficients, they used a template attack followed by a key enumeration technique that they argue is simpler than belief propagation used in SASCA to recover the secret key (during key generation) or the session key (during key encapsulation).

Karatsuba multiplication. Bock *et al.* [BBB⁺24] exploit that Kyber’s incomplete NTT causes each secret coefficient to be used multiple times, unlike when performing a complete NTT. This profiled attack targets a single trace and assumes a known ciphertext, therefore working in static and ephemeral setups. The main idea of the attack is to profile each incomplete NTT multiplication, which depends only on the public ciphertext and two secret coefficients, which are relatively small. This work avoids machine learning techniques to avoid complexity and combines correlation matching with the online template attack approach [BCP⁺14, BCP⁺17]. The authors further discuss the impact of changing the number of templates on the success rate of secret key recovery.

4.1.2 Leakage during NTT

Primas *et al.* [PPM17] presented the first single-trace attack on lattice-based schemes, specifically targeting the electromagnetic emanation leakage from NTT. To recover the full secret key, they use SASCA, which consists of three steps: template matching, belief propagation, and lattice decoding. For template matching, they used about 100 million traces. As their attack uses only a single trace after profiling, it is also effective against masked implementations.

This attack was further explored by Pessl and Primas [PP19], who focused on the practicality of this attack on state-of-the-art constant-time implementations. They reduced the number of required templates by performing template matching on the loading and storing of butterfly inputs/outputs instead of on modular multiplication and by constructing

templates only for Hamming weights rather than all possible values. They also improved the performance of belief propagation by applying existing methods for cyclic factor graphs. With these optimizations, only 213 templates were needed, compared to the previous 100 million. However, the attack was changed to target only the session key rather than the full secret key. To prevent this attack in both cases, the authors propose the randomization of the NTT computation using shuffling of processed coefficients or inserting dummy operations.

4.1.3 Leakage during message encoding and decoding

In an ephemeral key setup, leakage during message encoding and decoding is used only to retrieve the session key; as for secret key recovery from the message, multiple traces corresponding to chosen ciphertexts are required, as described earlier. Sim *et al.* [SKL⁺20] were able to recover the established session keys in multiple lattice-based schemes by targeting the leakage during message encoding. After identifying specific points of interest, they classified these points into two groups using the clustering algorithm. Their attack achieved a 100% success rate on Kyber. Furthermore, they demonstrated the broader applicability of their attack to other lattice-based schemes.

Table 4: Summary of side-channel attacks on lattice-based KEM schemes in ephemeral key setup.

Target	Ref.	HW/SW	Technique	Adversarial	Leaked	Possible Countermeasures
Pointwise Multiplication	[YRZ ⁺ 23]	SW	TA	NI + D	SK	specific masking [RPBC20], shuffling
	[BBB ⁺ 24]	SW	TA	I + D	SK	shuffling
NTT	[PPM17]	SW†	SASCA	NI + D	SK	constant-time, shuffling, dummy operations
	[PP19]	SW†	SASCA	NI + D	M	shuffling, dummy operations
Message Encode/Decode	[SKL ⁺ 20]	SW	TA	NI	M	shuffling

HW/SW: † - masked

4.2 Attacks on Code-based Schemes

This section presents existing attacks against code-based KEM schemes in an ephemeral key setup categorized according to the targeted operation. All attacks are summarized in Table 5, following the same structure as described in Section 3.1. The only difference is that, for the ephemeral key setup, we indicate whether the secret key (SK) or the message (M), representing the session key, was leaked.

4.2.1 Leakage during pointwise multiplication

In the McEliece cryptographic scheme, the Syndrome Decoding Problem (SDP), which is computationally infeasible to solve in polynomial time, can be degraded to the Integer Syndrome Decoding Problem (N-SDP) if an attacker obtains the ciphertext (syndrome) value in $\mathbb{N}$. This can be achieved through side-channel information, allowing the attacker to retrieve the original message. In [CDCG22], Colombier *et al.* recovered the message using the same attack in a hardware implementation of McEliece with packed matrix-vector multiplication. They use side-channel analysis in a profiling set-up with random forest to obtain a faulty syndrome in $\mathbb{N}$ from the Hamming weight information leaking during the multiplication of the H matrix elements with the error vector elements, e , in the encryption process. Then, it is possible to recover the message from the recovered noisy syndrome by leveraging a simple, efficient dot product and N-SDP.

The authors propose using masking to generalize the error vector weight pattern to reduce this attack.

4.2.2 Leakage during message encoding and decoding

Goy *et al.* [GMGL24] and Baisse *et al.* [BMG⁺24] targeted the leakage of decoders in HQC, and using SASCA, they were able to execute single-trace attacks on this scheme. Goy *et al.* extended their previous research [GLG22] by enabling a single-trace attack on the session key. In this work, they exploit side-channel leakage from both the Reed-Solomon decoder and encoder. They demonstrate the feasibility of combining leakage from the decoder and encoder during re-encryption, allowing for successful key recovery even in environments with higher noise levels. Additionally, they reveal that current countermeasures, such as masking or shuffling, are ineffective against their attack and propose a modification to the RS encoder algorithm to enable full shuffling. Building on this approach, Baisse *et al.* targeted the Reed-Muller decoder, which precedes the Reed-Solomon decoder, thereby enabling recovery of the secret key through the use of specially crafted ciphertexts.

Table 5: Summary of side-channel attacks on code-based KEM schemes in ephemeral key setup.

Target	Ref.	HW/SW	Technique	Adversarial	Leaked	Possible Countermeasures
Pointwise Multiplication	[CDCG22]	SW	DL-SCA	NI + D	M	masking
Message Encode/Decode	[GMGL24]	SW†‡	SASCA	NI + D	M	encoder modification + full shuffling
	[BMG ⁺ 24]	SW*	SASCA	I + D	SK	masking, shuffling

HW/SW: † - masked, ‡ - shuffled, * - simulated

5 Side-channel Attacks on DSA Schemes

The objective of attacks on digital signature schemes is to enable the forgery of signatures for any arbitrary message in the future. Consequently, recovering the full secret key is not always required. Studies, such as [RJH⁺18, CKA⁺21, KA21], have demonstrated that partial recovery of the secret key is often sufficient to achieve this goal. All attacks against DSA schemes are summarized in Tables 6 and 7, with the same structures as described in Section 3.1.

5.1 Attacks on Lattice-based Schemes

5.1.1 Leakage during pointwise multiplication

Similar to the lattice-based KEM scheme, lattice-based DSA schemes also involve pointwise multiplication with a secret polynomial, making them vulnerable to similar attacks.

Chen *et al.* [CKA⁺21] exploited leakage during the multiplication of public challenge with the secret key within the signature generation phase in Dilithium using correlation analysis. They identified two primary points of vulnerability: integer multiplication and subsequent modular reduction, corresponding to attacks on KEM schemes [KdG21, MWK⁺24]. However, because of Dilithium’s large modulus, the key guessing space is significantly increased, making the attack time-consuming. To reduce computational overhead, they proposed methods to combine both leakage sources to narrow the key guess space, successfully demonstrating key recovery on a Cortex-M4. A similar attack was presented by Karabulut and Azarderakhsh [KA25], who attacked the hardware implementation that complies with the new ML-DSA standard. They exploited the fact that the implementation uses zeroization, in accordance with [NIS01], to obtain the reference values required for the Hamming distance model. Kim *et al.* [KLH⁺20] showed an attack on sparse multiplication using deep neural network profiling. They recovered the masked coefficient of s_1 using 1024 profiles and 8000 attack traces. Wang *et al.* [WGL⁺24] showed a CPA attack on polynomial multiplication of an FPGA implementation of Dilithium.

They retrieved the partial key using 70000 traces. They also showed that using different PoI selection techniques can reduce the required traces. Steffen *et al.* [SLKG23] showed a CPA on the polynomial multiplication of the signing procedure using 66000 traces to recover the full secret key on a FPGA implementation of Dilithium.

A different attack on pointwise multiplication was proposed by Karabulut and Aysu [KA21], who targeted the unique floating-point multiplication in the Falcon scheme, presenting the first side-channel attack on this scheme. They used differential analysis of electromagnetic emanation leakage to independently recover the mantissa, exponent, and sign bits of each secret coefficient. By recovering these parts independently, they significantly reduced the combinatorial space. To address false positives arising during mantissa recovery, they proposed a novel extend-and-prune technique. Guerreau *et al.* [GMRR22] further improved this attack. They showed that it is not necessary to recover the complete mantissa, but only the 6 most significant bits to recover the secret key, as the relative error is conserved through an inverse Fourier transform, reducing the required number of traces. They also proposed a method to mitigate noise.

5.1.2 Leakage during NTT

The attacks of Primas *et al.* [PPM17], and Pessl and Primas [PP19] described in previous sections are general attacks on NTT, so they also apply to DSA schemes that use this transform. A more specific, experimentally verified attack against the NTT in a DSA scheme was presented by Kim *et al.* [KLH⁺20], who targeted the NTT in Dilithium using a deep learning-based single-trace attack. The attack specifically focused on the secret coefficients s_0 and s_1 during each stage of the NTT. The authors distinguished two cases: whether a coefficient is used as input for multiplication by a twiddle factor or not. When a coefficient is multiplied by a twiddle factor, sufficient leakage is produced for an MLP to accurately classify its value. Throughout the stages of the NTT, coefficients at indices 1 to 255 participate in multiplication by the twiddle factor and can thus be progressively recovered. Although the 0th coefficient does not directly participate in such multiplications, once all other coefficients are recovered, the leakage from the first case is sufficient to deduce its value as well. Steffen *et al.* [SLKG23] mentioned a profiled SPA on polynomial multiplication to recover the full secret key on an FPGA implementation of Dilithium, too.

5.1.3 Leakage during sampling

Guerreau *et al.* [GMRR22] demonstrated that simple power analysis can be used to extract partial information from the sampling process in the Falcon cryptographic scheme. Using this partial information, they applied the Hidden Parallelepiped attack, originally introduced by Nguyen and Regev [NR06], to successfully recover the full secret key. The primary source of leakage arises during a comparison of two 72-bit values, which determines whether 1 should be added to a sampled coefficient. This comparison is implemented using three 32-bit subtractions, with eight most significant bits set to zero. When the result of the subtraction is negative (indicating that 1 should not be added), these eight bits are set to ones, creating a significant Hamming weight gap. To counteract this vulnerability, Guerreau *et al.* proposed modifying the sampling algorithm to reduce the Hamming weight gap. The same leakage was further examined by Zhang *et al.* [ZLYW23]. They discovered that the covariance of the samples in the filtered signatures could reveal the secret, allowing for a significant reduction in both the number of required traces and the CPU processing time. By additionally applying power analysis using sign leakage, Zhang *et al.* achieved further reductions in trace requirements, thereby improving the efficiency and effectiveness of the attack.

5.1.4 Leakage during polynomial packing and unpacking

Coefficients of polynomials are serialized to byte arrays or deserialized from byte arrays. This process can leak coefficients, which can be critical when working with secret data. Marzougui *et al.* [MUTS22] exploited such leakage during the unpacking of the y -polynomial in Dilithium’s signing process. Their attack utilized an MLP model to distinguish between zero and non-zero coefficients from power traces. By filtering predictions to reduce false positives, mapping predictions into a system of linear equations, and applying a least-squares method, they derived solution candidates. Using these candidates, they solved an integer linear program to recover the secret coefficients s_1 , enabling signature forgery. Qiao *et al.* [QLZ⁺23] presented a template attack on the y -polynomial, which uses only public values during profiling. They used leaked information on the bit of the y -polynomial to reduce the M-LWE problem to Integer-LWE, which is solvable in polynomial time. They show that the same leakage can also be exploited in the masked implementation of Dilithium.

Another attack exploiting this leakage was presented by Wang *et al.* [WNGD23], who demonstrated that it could be exploited to attack the key unpacking phase in Dilithium’s signing process using deep learning techniques. Their attack is a single-trace attack, enabling the recovery of portions of the secret key. The remaining coefficients are then computed through post-processing, employing one of two proposed variants based on whether the t_0 coefficients are known.

5.1.5 Leakage during decompose

The decompose operation in Dilithium divides the coefficients of a polynomial into two parts: high bits and low bits. Magliore *et al.* [MGTF19] identified this operation as the most challenging to mask within the Dilithium scheme. The fact that leakage during this operation can lead to secret key recovery was demonstrated by Berzati *et al.* [BVC⁺23]. They used a template attack to detect instances where a coefficient of the polynomial w_0 , which represents the low bits resulting from decomposition, is equal to zero. Using this information, they construct a system of linear equations that they solve using the Least Squares Method to recover the secret key polynomials s_1 , which enable the forgery of signatures.

5.2 Attacks on Hash-based Schemes

To the best of our knowledge, only one side-channel attack has been published targeting hash-based schemes relevant to the SLH-DSA standard. Other attacks on these schemes, such as fault-injection attacks, fall outside the scope of this work.

5.2.1 Leakage during hashing

In [KGB⁺18], Kannwischer *et al.* investigated the DPA for XMSS, XMSS^{MT}, and SPHINCS, three hash-based signature (HBS) schemes. They analyzed the side-channel resistance of hash functions and SHA-256-based PRNG for XMSS and XMSS^{MT}. They used simulations in the Hamming weight leakage model, assuming that the intermediate values inside the compression function leak the Hamming weight of each byte separately. However, in actual implementations, the intermediate values often leak the Hamming weight of 32 bits at once. Kannwischer *et al.* suggested using partial DPA with more traces to compensate for that. They also analyzed the DPA attack on the BLAKE hash function in SPHINCS-256. They targeted the compression algorithm in the used PRF (BLAKE-256) and could recover a 32-bit chunk of one of the secret keys using the Hamming weight leakage model and 10000 electromagnetic traces. To prevent the attack on BLAKE-256, they suggested hiding the order of the Mix procedures.

Table 6: Summary of side-channel attacks on lattice-based DSA schemes.

Target	Ref.	HW/SW	Technique	Adversarial	Possible Countermeasures
Pointwise Multiplication	[KLH ⁺ 20]	SW†	DL-SCA	NI + D	-
	[CKA ⁺ 21]	SW	CPA	NI	masking, shuffling
	[KA21]	SW	CPA	NI	masking
	[GMRR22]	SW	CPA	NI	masking
	[SLKG23]	HW	CPA	NI	masking, dummy operations
	[WGL ⁺ 24]	HW	CPA	NI	masking, dummy operations
NTT	[KA25]	HW	CPA	NI	masking, dummy operations
	[KLH ⁺ 20]	SW	DL-SCA	NI + D	masking
	[PPM17]	SW†	SASCA	NI + D	constant-time, shuffling, dummy operations
Gaussian Sampler	[PP19]	SW†	SASCA	NI + D	shuffling, dummy operations
	[GMRR22]	SW	SPA	NI	sampler modification
Polynomial Pack/Unpack	[ZLYW23]	SW	SPA	NI	sampler modification, masking
	[MUTS22]	SW	DL-SCA	NI + D	masking
	[QLZ ⁺ 23]	SW	TA	NI	time delays, implementation optimization
	[WNGD23]	SW	DL-SCA	NI + D	masking, constant-weight encoding
Decompose	[BVC ⁺ 23]	SW	TA	NI + D	masking, shuffling, secret sharing

HW/SW: † - masked

Table 7: Summary of side-channel attacks on hash-based DSA schemes.

Target	Ref.	HW/SW	Technique	Adversarial	Possible Countermeasures
Hash Functions	[KGB ⁺ 18]	SW	DPA	NI	hiding

6 Combining SCA with Cryptoanalysis

The attacks discussed in earlier sections primarily target specific cryptographic schemes or extend their approach to similar schemes, focusing on recovering the complete secret through SCA. For more practical attacks, it could be more beneficial to explore how some additional small information obtained using a side channel can lead to reducing the complexity of theoretical cryptoanalysis attacks. These small additional information are referred to as *hints*. This approach has been investigated in several studies, notably by Dachman *et al.* [DSDGR20] for lattice-based schemes and by Horlemann *et al.* [HPR⁺22] for code-based schemes.

Dachman *et al.* developed a framework to evaluate the security of LWE with additional hints, for example, hints from side-channel attacks. They introduced the Distorted Bounded Distance Decoding (DBDD) problem and showed how the LWE problem can be reduced to it. They define four types of hints: perfect hints, modular hints, approximate hints, and short vector hints, which can be integrated into the DBDD instance and then reduced to a unique-SVP instance, which can be solved using lattice reduction. They showed that their approach can perform better than scheme-specific attacks while being more general. This was demonstrated in practice when showing the feasibility of a single-trace attack on Frodo, which was originally considered unfeasible [BFM⁺19], due to the large combinatorial space. They used the additional information collected using a template attack to construct approximate hints and, with their application, reduce the security level. Dachman *et al.* [DSGHK23] further extended their research with new methods to integrate hints into DBDD, May *et al.* [MN23] introduced a new methodology to efficiently combine an arbitrary number of perfect and modular hints, and Hermelink *et al.* [HSMP24] defined distribution hints to model probabilistic information.

Building upon the idea of Dachman *et al.* in [DSDGR20], Horlemann *et al.* [HPR⁺22]

extended the concept of integrating side-channel hints into cryptanalysis for code-based cryptography. Their work focuses on how small information leakages, such as knowledge of error positions, Hamming weights of error blocks, or partial message recovery, can reduce the complexity of solving the (n, k, t) -SDP through Information Set Decoding (ISD). They adapted classical ISD algorithms, including Prange's, Lee-Brickell's, and Stern's, to incorporate such leakages, demonstrating significant reductions in computational effort. Horlemann *et al.* also presented a general framework systematically evaluating the impact of side-channel information on the security of post-quantum cryptosystems, such as Classic McEliece, HQC, and BIKE. By translating concepts originally developed for lattice-based systems, they highlighted how partial information, even from practical side-channel attacks, can lower the work factor for decoding, potentially undermining the claimed security levels of these schemes. This emphasizes the importance of accounting for side-channel vulnerabilities both in theoretical cryptanalysis and in the design of secure cryptosystems.

These methods are also relevant when the additional information obtained through side channels does not directly lead to the recovery of secret data but instead reduces the complexity of the cryptanalysis. This is especially useful when the obtained information is incomplete or probabilistic, as in DL-SCA, where the accuracy of the classified data depends on the trained model's performance.

7 Side-channel Countermeasures

Building upon the analysis of SCA on PQC schemes in the previous sections, we now discuss potential countermeasures and assess their implementation costs in both software and hardware. This section is divided into three parts. First, we analyze masking countermeasures, as masking is a widely adopted technique to protect cryptographic implementations against side-channel leakage with formal security proofs against side-channel leakage. Second, we examine other countermeasures based primarily on randomizing computations to obscure side-channel leakages. Third, we discuss schemes designed with inherent side-channel resistance or those that minimize the implementation overhead of countermeasures.

The summarized findings are presented in Table 8 for software and Tables 9 and 10 for hardware implementations. For both software and hardware implementations, we focus on the performance overhead resulting from the application of countermeasures. Additionally, for hardware implementations, we also consider area overhead, which reflects the increase in usage of key FPGA resources such as lookup tables (LUTs), flip-flops (FFs), digital signal processing (DSP) blocks, and block RAMs (BRAMs).

7.1 Masking Countermeasures

Masking splits secret data into multiple randomized shares, with the masking order d determining the number of shares. Secure computations are then performed on these shares. Although masking is generally effective against differential analysis, with the masking order matching the analysis order, it is less efficient against single-trace attacks. In PQC schemes, two primary forms of masking are used: boolean masking, where the secret value x is represented as $x = x_1 \oplus \dots \oplus x_d$, and arithmetic masking, expressed as $x = x_1 + \dots + x_d$.

Masking linear operations (e.g., XOR, addition, multiplication) is straightforward. Each share can be processed independently, either sequentially (typically in software implementations), which results in reduced performance, or in parallel (typically in hardware implementations), which results in increased resource usage. The main complexity arises with non-linear operations (e.g., AND, division), which usually require secure recombination of shares. These operations require specialized masked algorithms, which remain an active area of research and optimization. Coron *et al.* [CGTZ23] observed

that the operations that are the most challenging to mask tend to be relatively simple in unprotected implementations, while operations typically targeted for optimization in unprotected designs incur lower overheads when masked.

When masking entire schemes, conversions between boolean and arithmetic masking are often necessary. For example, operations like sampling are more efficiently handled using boolean masking, whereas arithmetic masking is better suited for core arithmetic operations. These conversions typically impose substantial overhead and often become performance bottlenecks. The algorithms facilitating these conversions have been extensively studied, for example, in [Gou01, BBE⁺18, SH24, NDKV24].

For instance, in Dilithium, even the `decompose` operation alone necessitates such conversions, as part of the operation is more efficiently handled in the boolean domain and part in the arithmetic domain. Performing these conversions incurs less overhead than implementing the entire function in a single domain. This holds for the standard `decompose` function. In contrast, Coron *et al.* [CGTZ23] proposed an alternative algorithm that achieves the same functionality while being optimized entirely in the arithmetic domain using newly introduced gadgets. The `decompose` operation remains recognized as the most challenging part of Dilithium to mask due to its dependence on conversions, substantial overhead, and central role in the signing process.

7.1.1 Masking lattice-based schemes

Following the success of lattice-based schemes in the NIST competition, significant research has been devoted to developing masked implementations. Numerous studies propose masked algorithms, evaluate their efficiency in software and hardware, and explore optimizations and their implications for fully masked designs.

Kyber. Kyber has received the most attention among lattice-based schemes. Beyond the arithmetic operations, which are relatively trivial to mask, two additional components require protection: the message decoder and the FO transform. The message decoder, in particular, has been a major focus. It was first masked by Reparaz *et al.* [RSRVV15] in 2015 in their masked hardware implementation of the R-LWE-based scheme. Their masked decoder was proposed to be resistant to first-order SCA. It takes as input an arithmetically masked coefficient and produces as output a boolean masking of a decoded message bit. Their proposed algorithm works in multiple iterations and brings an additional small probability of failure. The area utilization of the masked implementation increased by less than 20% but at the cost of reduced performance, which achieved only about 1/3 of the unprotected implementation. Oder *et al.* [OSPG16] proposed an algorithm based on modulus switching and arithmetic to boolean conversion to eliminate iterations and increase failure rate. Another improved decoder, which does not increase the probability of decoding failure, was proposed by Zijlstra *et al.* [ZBT19]. Their algorithm does not work in multiple iterations, thus the performance was not that much reduced, however, the required area increased by 3.7x in LUT usage. Bos *et al.* [BGR⁺21] extended the work to higher-order conversion again based on bit-sliced binary search after applying arithmetic to boolean masking.

The FO transform is another important part to secure. This includes protecting all the steps during re-encryption and then comparison of ciphertexts. Oder *et al.* [OSPG16] presented a first-order hash-based comparison, where a share is subtracted from the public ciphertext and the hash of this result is compared with the hash of the second share. This approach was found insecure, as analyzed in the attack section by Guo *et al.* [GJN20]. Bos *et al.* [BGR⁺21] presented a new algorithm for comparison between masked uncompressed ciphertext and public compressed ciphertext.

Fritzmann *et al.* [FVBBR⁺21] presented a masked co-design implementation of Kyber, where critical non-linear operations were implemented in hardware to remove performance bottlenecks. For Kyber, these non-linear operations include hashing, binomial sampling,

A2B/B2A conversions, and compression.

Some masked hardware implementations of Kyber were presented by Zhao *et al.* [ZPM⁺23]. In their work, they implemented two types of masking, arithmetic and hybrid, and evaluated their effectiveness against the differential analysis attack described in the previous section. In arithmetic masking, the secret key is divided into shares $s_0 = sk - R$ and $s_1 = R$, where R is a random value. All subsequent operations are performed separately for each share. Combined masking combines multiplicative masking during polynomial multiplication with arithmetic masking afterwards. Initially, multiplicative masking is applied to protect polynomial multiplication, after which the coefficients are converted to arithmetic masking. The remaining operations then proceed as in pure arithmetic masking.

Dilithium. Migliore *et al.* [MGTF19] presented the first software masked implementation of Dilithium, using the work of Barthe *et al.* [BBE⁺18], who presented the first masked implementation of the lattice-based signature scheme. They implemented multiple secured gadgets for Dilithium-specific operations, such as decomposition, which they identified as the most challenging operation to mask, `makehint`, which derives hint bits using the decomposition to assist in reconstruction during signature verification, and signature rejection. To improve the efficiency of their implementation, they proposed changing the Dilithium modulus so that it is power-of-two, making the decomposition simpler.

Further advances were made by Azouaoui *et al.* [ABC⁺23] and Coron *et al.* [CGTZ23], who improved the masked gadgets necessary for Dilithium operations. Azouaoui *et al.* also suggested additional areas within Migliore *et al.* implementation that required masking while identifying some parts that did not need it.

The first hardware-masked implementation of Dilithium was recently presented by Raj *et al.* [RRCC24]. Their work closely followed the masked software gadgets proposed by Conor *et al.* without specific optimizations using parallelization. As a result, their design incurs a high performance overhead while maintaining a relatively low area overhead.

Falcon. Karabulut and Aysu [KA24] presented the first hardware masking scheme for floating-point multiplication used in Falcon. They implemented three variants with different area/performance trade-offs, with the balanced variant having an area overhead of almost 17x and a performance overhead of almost 13x. This countermeasure directly mitigates leakages in floating-point multiplication exploited by [KA21, GMRR22].

7.1.2 Masking code-based schemes

Code-based schemes have received considerably less attention. However, with the recent standardization of HQC, there is a growing need to study their masked implementations. To date, only a few such efforts have been reported.

McEliece. Chen *et al.* [CEvMS16] introduced the first masked implementation of McEliece on FPGA, providing a foundation for subsequent research into masked implementations of code-based schemes. Their approach applied masking to the syndrome computation, which was straightforward, but the decoding algorithm posed more complexity. Although this work is relatively old, research into masking code-based schemes has been limited.

BIKE. Demange and Rossi [DR24] recently presented a provably masked software implementation of BIKE was recently presented by Demange and Rossi [DR24]. Their work introduced the first high-order masked implementation of a code-based scheme. They opted to implement masking without mask conversion, arguing that the conversions were inefficient despite the significant body of research on the topic. Consequently, their implementation stayed within boolean masking throughout the entire computation. Since code-based masking is not as developed as lattice-based masking, they proposed several new gadgets, including the Black-Gray-Flip (BGF) Decoder, Fisher-Yates generation of sparse polynomials, and polynomial inversion.

HQC sampling. Spyropoulos *et al.* [SVP⁺24] proposed a masked sampling algorithm for HQC, which secures Sendrier’s modified Fisher-Yates method [Sen21], originally developed to address the attack by Guo *et al.* [GHJ⁺22]. The core of this sampling process is modular reduction, for which they implemented masked Barrett reduction. They developed two variants: one that operates entirely on boolean-masked values and another that incorporates conversions to arithmetic masking. Their findings, similar to those observed with Dilithium’s `decompose` function, suggest that introducing masking conversions can lead to more optimized solutions in some instances. They further argue that Demange and Rossi’s work could benefit from such conversions to achieve better optimization.

Table 8: Summary of overheads in software masked implementations.

Ref	Family/Scheme	Number of Shares	Performance overhead
[OSPG16]	R-LWE	2	5.7
[BGR ⁺ 21]	Kyber	2/3/4	3.5/50.3/130.9
[MGTF19]	Dilithium	2/3/4	5.7/11.8/28.3
[ABC ⁺ 23]	Dilithium	2/3/4	6.4/18.5
[CGTZ23]	Dilithium	2/3/4	43/98/153
[DR24]	BIKE	2/3/4	5.8/14.2/24.4
[SVP ⁺ 24]	HQC	2	23.4/16.3*

* with boolean to arithmetic mask conversion

Table 9: Summary of overheads in hardware masked implementations.

Ref	Family/Scheme	Area overhead	Performance overhead
[RSRVV15]	R-LWE	1.18	3.22
[ZBT19]	R-LWE	2.73	1.30
[ZPM ⁺ 23]	Kyber	1.20	1
[KNAH22]	Kyber	1.34	1.09
[FVBRR ⁺ 21]	Kyber*	1.44	3.95
[KA24]	Falcon	10.4	11.1
[RRC24]	Dilithium	1.31	50
[CEvMS16]	McEliece	7.8	4.35

* co-design implementation.

7.2 Other Countermeasures

7.2.1 Protecting lattice-based schemes

Shuffling. Shuffling randomizes the order of operations that are not dependent on each other. For example, it is often used during polynomial multiplication in lattice-based schemes. Zijlstra *et al.* [ZBT19] proposed two shuffling methods for NTT and pointwise multiplication in their R-LWE scheme implementation. The first method uses a linear feedback shift register (LFSR) where a normal loop counter is replaced by an LFSR, generating a pseudorandom sequence. The other method uses the permutation network to generate the sequence. When comparing these two methods, the LFSR offers lower area utilization and higher throughput at the cost of a lower added entropy. Other countermeasures based on shuffling were proposed by Chen *et al.* [CMJ23]. They introduced two novel schemes: coefficient index randomization and NTT network randomization to protect against existing simple analysis and template attacks. The proposed countermeasures

exhibit minimal hardware overhead, requiring only a 9% increase in LUT usage. Their effectiveness was experimentally validated against attacks such as those by Xu *et al.* [XPR⁺22] and Aysu *et al.* [ATT⁺18]. Xu *et al.* [XWT24] proposed a novel hardware-friendly shuffling countermeasure for the NTT/INTT and polynomial multiplication module. To prevent collisions in memory access during NTT/INTT, they use two different permutation sets that are shuffled. Using this countermeasure, they were able to protect against CPA attacks, showing, with practical measurements, that with unprotected design, the implementation was attacked with a minimum of 900 traces. In contrast, the protected implementation did not show a correlation with 100k traces. Implementing the countermeasure led to only an 8.7% reduction in hardware efficiency. The detailed results are presented in Table 10, with the unprotected reference results taken from the work of Xing and Li [XL21].

Randomization. Jati *et al.* [JGCS24] applied multiple countermeasures based on randomization to their hardware implementation of Kyber. These countermeasures include inserting random delays between operations, making the attacks more difficult, as they require trace synchronization, address, and operations randomization, effectively shuffling the order, and fast hash functions for detections of fault injection. As these mechanisms relied on the efficiency of the random generator, they also implemented a true random generator based on ring oscillators. In multiple attacks against hardware implementations, the randomization of register values by performing a dummy operation was proposed as a possible countermeasure. This randomization effectively hides the reference value, which is required in the Hamming distance model, making the attack significantly more difficult. It is a very lightweight countermeasure and was implemented, for example, in [RBVB23].

Redundant number representation. To the multiple countermeasures already discussed by Zjilstra *et al.* [ZBT19], Zjilstra *et al.* also added a redundant number representation countermeasure. This involves adding random multiples of the modulus q to the secret coefficients, effectively randomizing them.

Hiding. Kamucheka *et al.* [KNAH22] implemented protected Kyber for the lowest security level. They applied a novel approach to hiding, using parallel processing, pipelining, and loop unrolling. In addition, they also applied masking techniques to enhance the security of their design. Their implementation was demonstrated to be secure against SPA and DPA.

Duplication. Moraitis *et al.* [MJB⁺24] protected reference implementation of Kyber [XL21] using duplication and clock randomization countermeasures. The duplication countermeasure simply uses two cores that run on the same input data but with different keys, while each core is driven by different randomized clocks. The authors evaluated the effectiveness of these measures by performing a deep learning-based attack aimed at recovering the encrypted message, representing the session key. The experimental results revealed that while clock randomization alone posed challenges for the attack, it remained feasible. However, the combined use of clock randomization and duplication rendered the attack infeasible. Despite their effectiveness, these combined countermeasures impose a significant cost, nearly doubling resource utilization and reducing the actual operating frequency.

7.2.2 Protecting hash-based schemes

Fake signature compensation. Huang *et al.* [HLL⁺24] focused on leakage caused by storing a signature. In their implementation, after a valid signature was generated, it would be stored in RAM, which would increase power leakage and indicate which signature is valid. They proposed to split the RAM into two parts for proper signatures and fake signatures, which would allow the same power leakage, not depending on whether the signature was true or fake.

7.2.3 Protecting code-based schemes

Algorithm change. Schoffel *et al.* [SFW24] focused on a new updated, more secure sampling algorithm based on Sendrier’s modified Fisher-Yates approach [Sen21], which was proposed by the authors of HQC in response to attack [GHJ+22]. Schoffel *et al.* implemented this algorithm and optimized it using the cross-layer approach to enable efficient hardware implementation. From Table 10, it can be seen that their optimizations significantly increase the number of operations per second while offering slightly lower area utilization.

Table 10: Summary of overheads in hardware implementations of countermeasures.

Countermeasure	Ref	Family/Scheme	Area overhead	Performance overhead
permutation/LFSR shuffling	[ZBT19]	R-LWE	6.59/2.21	1.47/1.28
shuffling	[CMJ23]	Kyber	1.07	-
shuffling	[XWT24]	Kyber	1.11	-
dummy operation	[RBVB23]	Kyber	1.06	-
randomized clock and duplication	[MJB ⁺ 24]	Kyber	1.98	-
fake signature compensation	[HLL ⁺ 24]	SPHINCS ⁺	-	-
constant-time sampling	[SFW24]	HQC	0.81	0.04

7.3 Schemes Designed with Countermeasures in Mind

From the summary tables, it is evident that countermeasures, particularly masking, come with a significant overhead. As a result, there has been a growing interest in designing cryptographic schemes with side-channel resistance in mind from the outset.

One such example is Raccoon, proposed by Del *et al.* [dPKPR24], which was submitted to the NIST standardization process for additional digital signature schemes. However, it did not advance beyond the first round. Although NIST did not provide detailed reasons for the exclusion, the selection process prioritized algorithms that offered greater diversity in design approaches and security assumptions. The similarity of the Raccoon with Dilithium, which was already standardized, may have contributed to its exclusion despite its strong side-channel resistance.

Another example is Polka, introduced by Hoffmann *et al.* [HLM⁺23], which incorporates design choices to enable efficient side-channel-protected implementations. For example, it avoids the FO transform, shown in the previous sections to be a significant source of leakage, and introduces ciphertext randomization through the use of a dummy ciphertext. This makes decryption ephemeral and reduces attack vectors, especially in contrast to static key setups, which are easier and more often targeted. While this approach reduces the complexity of implementing side-channel protections, it is based on the relatively new Learning with Physical Rounding problem, which still requires further study and validation. These proposals highlight promising research directions, but the fact that Raccoon did not pass the first round of the NIST’s additional signature scheme evaluation suggests that such designs are not yet mature enough for standardization.

8 Discussions

This section discusses our overall observation after considering an enormous range of work on hardware and software. These include high-level overviews of the attacks analyzed, general understandings, and reflections that can be considered as takeaways or open questions that can be appealing to investigate.

8.1 Cross-Scheme Applicability of Attacks

Due to the similarities among post-quantum schemes, particularly within cryptographic families, such as the use of NTT-based multiplication in lattice-based schemes, or across schemes with the same use cases, such as the use of the FO transform in many KEM schemes, successful attacks on one scheme can often inform potential vulnerabilities in others. This opens up new research avenues for targeting analogous operations in similar schemes. A compelling example is Ueno *et al.* [UXT⁺21], who demonstrated an attack on FO transform applicable to multiple schemes, including Kyber and HQC.

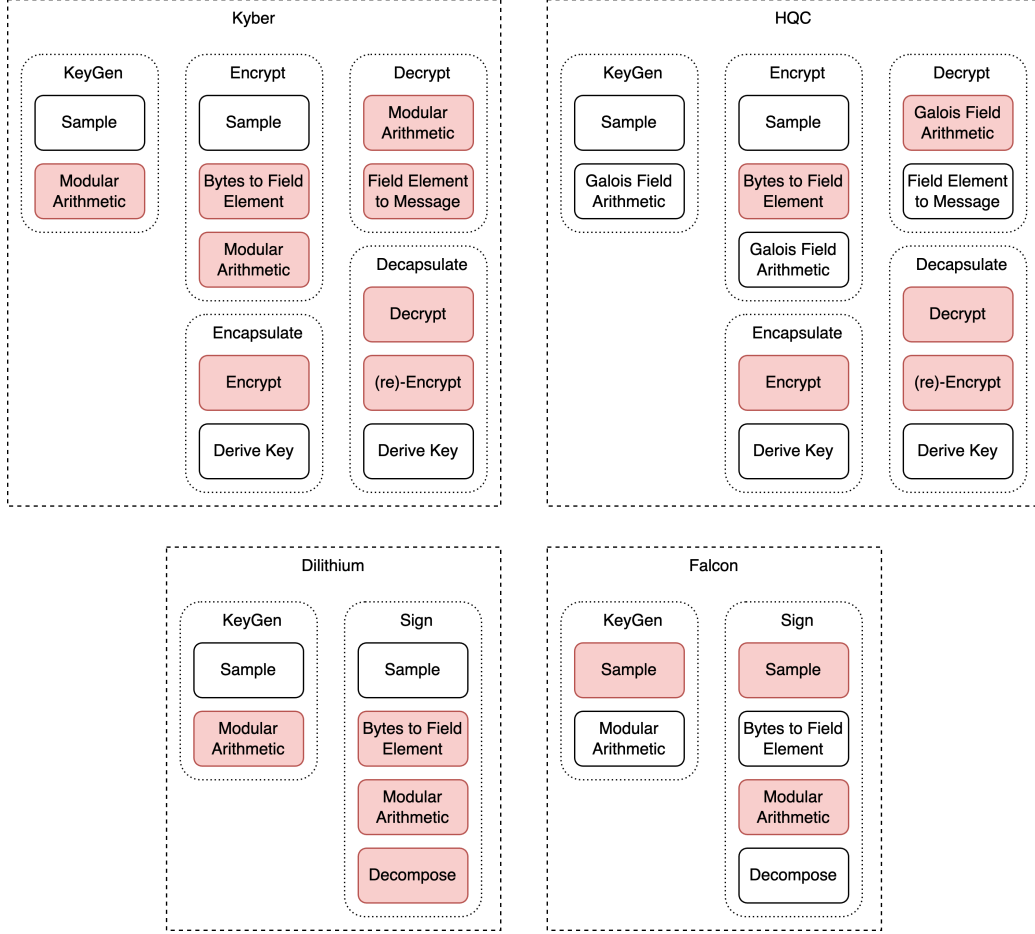


Figure 4: Vertical comparison of secret-dependent operations in different schemes.

To illustrate potential research gaps, Figure 4 compares the KEM schemes Kyber and HQC and the DSA schemes Dilithium and Falcon, breaking each into high-level secret-dependent operations. Red highlights indicate components previously targeted by side-channel attacks. In the Kyber-HQC comparison, an asymmetry stands out: while Kyber’s modular arithmetic has been widely targeted, HQC’s corresponding Galois field arithmetic remains unexploited. One possible explanation lies in the number of leakage points, while Kyber uses matrix-vector multiplication, where the same secret value is multiplied with multiple public values, thus offering multiple points of leakage, HQC uses vector-vector multiplication, thus offering fewer points of leakage. This suggests a potential overlooked target in HQC implementation, i.e., vector multiplication, and offers promising directions for further investigation.

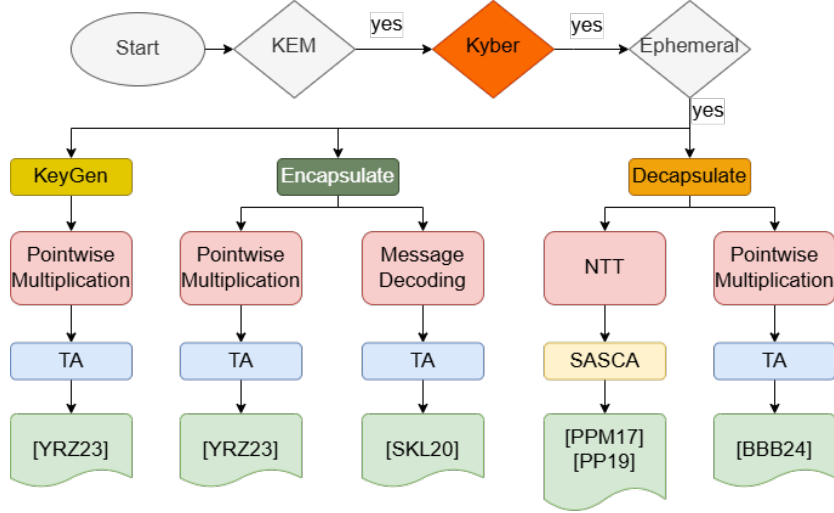


Figure 5: A guideline to evaluate an implementation of Kyber with ephemeral key setup against all explored attacks so far.

A similar pattern appears between Dilithium and Falcon. Despite a functional overlap, some Falcon operations, like the decompose function, remain untargeted, even though analogous attacks have succeeded in Dilithium. This indicates that Falcon’s decomposition may also be vulnerable and that insights from Dilithium-focused research could guide future explorations. The same applies to the Bytes to Field Element operation, which may represent another unexplored target.

These overlooked components point to opportunities for cross-scheme generalization of attacks. As such, these underexplored areas provide clear and actionable directions for future side-channel analysis. This question can be extended even to schemes from different families. An example can be Dilithium and SPHINCS⁺, both signing algorithms but from different families.

8.2 Evaluation Guidelines

One of the benefits of analyzing a large body of work and identifying shared characteristics across different schemes is the possibility of deriving practical guidelines for evaluating cryptographic implementations. Based on our study of many attacks, both in software and hardware implementations, we outline how this knowledge can help guide side-channel evaluation efforts in practice. This subsection presents two examples showing how the information gathered in Sections 3, 4, and 5, along with the background from Sections 2.3 and 2.5, can be used to develop targeted evaluation strategies. These examples can serve as starting points for practitioners testing or certifying implementations against side-channel attacks.

There are two important points to consider:

1. **Cross-Applicability of Attacks:** Although some attacks are demonstrated in either software or hardware contexts, many of them can be adapted across different platforms. Therefore, we treat an attack demonstrated in one setting as relevant for both when evaluating implementations.
2. **Multiple Techniques for the Same Target:** It is common for the same vulnerable operation to be attacked using different side-channel techniques (e.g., power analysis,

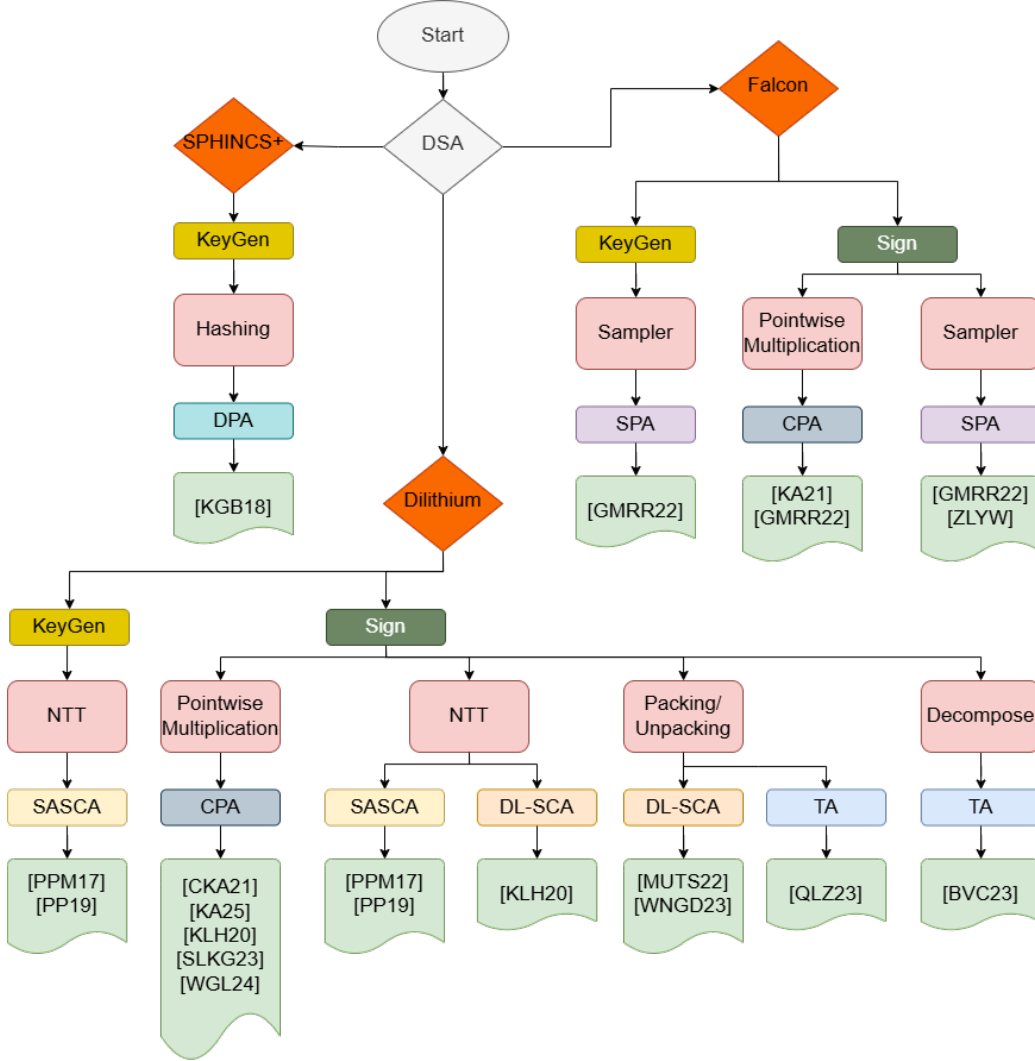


Figure 6: A guideline to evaluate implementations of DSA schemes (Dilithium, Falcon, SPHINCS+) against all explored attacks so far.

timing, or template attacks). Evaluators should consider which techniques are most effective for each leakage point.

As a concrete example, Figure 5 illustrates a guideline for evaluating implementations of the Kyber KEM in an ephemeral key setting. The flowchart highlights three core procedures: key generation (KeyGen), Encapsulate, and Decapsulate (each in a different color). Each procedure includes specific subroutines (referred to as target operations in Section 3 to Section 5). The flowchart shows those subroutines (target operations) exploited using SCA to retrieve the secret information, including the secret key or long-term session key (message). Moreover, for each target subroutine (shown in light-red color), it specifies the effective attack technique (e.g., TA and SASCA, each shown in a different color to emphasize which attack technique was used for which target operations) and the corresponding literature (shown in light-green color). This structure provides a practical attack-oriented perspective for evaluators: instead of assessing the implementation as a black box, one can focus on specific, well-known leaky components using the most effective

attack methods reported to date.

The second example, shown in Figure 6, provides a guideline for the three signature schemes considered in this work: Dilithium, Falcon, and SPHINCS⁺. This flowchart further illustrates the diversity of attack surfaces and provides a comparative overview of known vulnerabilities and techniques across schemes. In this case, the presence of multiple applicable side-channel techniques for the same target is even more evident. For example, the NTT subroutine appears in both Dilithium’s key generation and signing procedures and has been exploited using both SASCA and DL-SCA. Similarly, the sampler, a critical component in Falcon, has been attacked using SPA and template attacks in different works. This shows how the same computational block can be exploited through various channels, depending on the implementation details and attacker capabilities.

Another key insight from this flowchart comes from comparing it with the explored and unexplored secret-dependent operation in Figure 4. It highlights unexplored but potentially leaky operations. For example, packing/unpacking and decomposition in Dilithium have been studied, but equivalent operations in Falcon remain uninvestigated. Similarly, some techniques, like CPA and SPA, are applied to specific components in Falcon but have not yet been systematically explored in Dilithium, despite functional similarities, which should be taken into account for a comprehensive evaluation.

These gaps suggest potential research directions in the current literature and offer clear directions for future evaluation efforts. Practitioners and certifying bodies as good as the research community should be aware not only of the known attack paths but also of those underexplored components, particularly when similar operations have already been shown to leak in related schemes. As a result, this comparative flowchart alongside Figure 4 serves as a practical tool for known attacks and a heuristic guide for identifying high-risk areas that deserve further attention.

8.3 Targets Ranking

This section ranks various cryptographic operations on the basis of their susceptibility to side-channel attacks. The targets are presented in an order that reflects their practical relevance and potential impact. Note that we focus only on general targets across multiple schemes. Although scheme-specific operations, such as `decompose` in Dilithium, can also present valuable points of attack, they are not considered here due to their limited applicability.

1. **FO Transform:** Attacks targeting the FO transform offer a wide range of possibilities, as leakage can occur at multiple points and is not tied to a single operation. Although such attacks typically require interaction with the target device and the construction of oracles, thereby increasing the attack complexity, they can be extremely powerful. In re-encryption scenarios, the attacker may profile directly on the device, and success can sometimes be reduced to distinguishing between just two hypotheses, making this attack vector particularly strong. As highlighted in [ABH⁺22, HLM⁺23], the FO transform introduces numerous leakage points that can be exploited. However, this attack vector is only applicable to KEM schemes utilizing the FO transform.
2. **Pointwise Multiplication:** Pointwise multiplication is one of the most critical and widely exploitable targets. In most cases, the attacker does not need to interact with the device beyond collecting traces. Many schemes, such as Kyber and Dilithium, involve repeated multiplication of the same secret vector with different public matrix rows within a single operation. This repetition enables horizontal attacks even from a single trace, which is particularly dangerous in ephemeral contexts. Furthermore, hardware implementations often reuse components for both pointwise multiplication and NTT/INTT operations to optimize resource usage. This increases vulnerability by exposing additional information, such as known reference values [RBVB23].

3. **Encode/Decode:** Encode and decode operations can be divided into two types: those that involve the session secret (as in KEM) and those that involve long-term secret keys (primarily in DSA). When targeting the session secret, the risk is generally lower than for pointwise multiplication since multiple traces are usually required to recover the secret key. In addition, these operations typically occur only once per session, which limits the number of leakage points.
4. **NTT:** The Number Theoretic Transform (NTT) is a valuable target, as it directly processes secret data. However, its relevance is limited to schemes that actually utilize the transform. In ephemeral settings, practical attacks, especially those relying on SASCA, may be hindered by the complexity of the operation. However, as shown in [XPR⁺22], if maliciously designed ciphertexts are used, attacks on NTT can extract secret keys with only a few traces.

8.4 PQC Side-Channel Attack Remarks

The following recurring patterns and insights have been identified by reviewing and analyzing the broad range of side-channel attacks on different PQC implementations. These observations highlight key challenges and considerations that must be addressed when designing and evaluating PQC implementations.

- **Chosen-ciphertext assisted SCA pose a serious threat:** Chosen inputs can significantly enhance attacks' effectiveness by amplifying the secret's leakage. Chosen-ciphertext attacks can cause the scheme to lose its IND-CCA security, enabling the recovery of secret keys. For instance, [XPR⁺22] demonstrated an approach that requires the smallest number of chosen ciphertexts, raising the possibility of applying their methodology to other attack scenarios.

In lattice-based schemes, chosen inputs are primarily utilized to maximize Hamming weight or Hamming distance, thereby amplifying leakage, whereas, in code-based schemes, they help identify points of change to exploit vulnerabilities. Additionally, chosen inputs play a critical role in reducing noise in parallel hardware executions and determining reference values essential for the Hamming distance model by setting non-target values to zero. These strategies demonstrate the substantial potential of chosen inputs to optimize the precision and efficiency of side-channel attacks.

- **SCA on software implementations are applicable to hardware implementations:** Several studies, including Guo *et al.* [GJJ22], applied the same attack to both software and hardware implementations. Rodriguez *et al.* [RBVB23] and Ji *et al.* [JWN⁺23], adapted attacks on software implementations ([MWK⁺24] and [WND23] respectively) to hardware implementations, demonstrating that hardware implementations are also vulnerable to similar types of leakage.

However, side-channel setups for hardware implementations are typically characterized by lower SNR⁸, resulting in the attacks requiring a larger number of traces or the use of chosen ciphertexts, as proposed by Wang *et al.* [WCCL24], to mitigate noise. In addition, the evaluation of hardware often faces challenges due to the lack of reference implementations, requiring attackers to first implement the schemes themselves before attempting to exploit them.

Since the first power attacks against FPGAs [ÖOP03, SÖP04], many of the same challenges persist as discussed at that time. Essentially, the resistance of hardware implementations against side-channel attacks is still much less explored than for software implementations.

⁸Due to parallel execution and hardness of building setups for trace collection.

- **Optimizations can introduce SCA vulnerabilities.** While many implementations focus on optimizing for performance or hardware resource utilization, this can come at the cost of reduced resistance against SCA. For example, the attack [BCM⁺23] targets key generation that should be executed only once, but because in some implementations, only the seed is stored, the key generation can be run multiple times.

Another example is NTT, where, for instance, Kyber stores secret keys in the NTT domain, while Dilithium does not. However, the work [QLZ⁺23] suggests storing it also for Dilithium as it removes the attack on NTT during the signing procedure. Nevertheless, works like [ADP18] show that storing coefficients in the NTT domain might allow different types of attacks.

- **Deep learning-based SCA can tackle common countermeasures:** Deep learning-based SCA, as analyzed in prior sections, can defeat even complex countermeasures, at least in the academic context in which the SNR is usually moderate. These attacks often succeed in extracting critical information—whether full secret data or partial data, such as in oracle-based attacks—leaving traditional masking countermeasures largely ineffective. Furthermore, attacks targeting masked and shuffled implementations have been demonstrated (e.g., [WCCL22, BNGD23]).⁹ Our limited understanding of neural network capabilities further complicates the development of effective defenses, with key unresolved challenges highlighted in [PHJ⁺17]. These challenges underscore the critical need for further investigation into how neural networks interact with countermeasures and the development of novel strategies to counteract these advanced attack methodologies.
- **Both static and ephemeral setups should be considered for SCA evaluation:** While the ephemeral key setups are generally recommended, as seen in protocols like TLS 1.3 [SFG24], attacks in static key protocols remain relevant. This is because many side-channel attacks require only a small number of traces to recover the secret key, making them potentially effective even in ephemeral settings. For instance, retransmissions or implementations that temporarily cache the secret key could enable these attacks. Moreover, guidance like [SFG24] is only a recommendation and may not be strictly followed in all cases. On the other hand, ephemeral key setups are not immune to SCA; there are multiple attacks possible in the ephemeral key setup that pose significant threats, challenging the assumption that ephemeral setups are a sufficient defense against SCA.

8.5 Open Areas for Research

The transition to post-quantum cryptographic schemes is still in its early stages. Through the analysis conducted in this work, we noticed several areas that seem to need further research.

- **Broader focus of SCA:** A noticeable trend in the literature is the concentrated focus on lattice-based schemes, particularly Kyber, which is understandable given its role in the NIST standardization process. However, expanding protection efforts to other schemes is important, especially as NIST continues the ongoing standardization process for additional signature algorithms [NIS22]. For example, HQC, recently selected for standardization, has received significantly less attention in side-channel security research, highlighting the need for a more balanced exploration across the full spectrum of PQC candidates.

⁹The works using deep-learning techniques usually use setups resulting in datasets with high SNR. While the power of neural networks for handling noise has been shown in general, it still needs to be confirmed for more noisy setups executing PQC algorithms.

- **Cross-Scheme Applicability of Attacks:** As already discussed in Section 8.1, due to structural and functional similarities among post-quantum cryptographic schemes, vulnerabilities identified in one scheme often suggest potential weaknesses in others. However, specific components remained underexplored despite being functionally similar to previously exploited ones. This opens up opportunities to extend existing attack strategies to related schemes by targeting similar operations.
- **Combination of SCA with cryptoanalysis:** While there are already studies combining the SCA with cryptoanalysis, as described in Section 6, we believe that this area offers more opportunities for research. For example, it calls for a deeper analysis of the trade-offs between the required number of traces, signal-to-noise ratio, and required time for offline computation. Another path could be to extend the research for remaining families, such as multivariate, as the main research was on lattice- and code-based schemes, with code-based extending the lattice-based.
- **Effective countermeasures against SCA for PQC:** Although many side-channel attacks have been proposed against PQC schemes, far fewer studies have focused on the practical protection of implementations. Most of the existing work proposes theoretical countermeasures with limited or no experimental validation. However, experimental evaluation is essential not only to confirm the effectiveness of proposed mitigation but also to understand their performance and resource costs in real-world scenarios.

As discussed in Section 7, most protected implementations are concentrated around Kyber and Dilithium. There is a clear lack of side-channel-protected implementations for the remaining standardized schemes. Furthermore, the impact of higher-order masking in hardware has not yet been explored. While software implementations are important, investigating the area/performance trade-offs in hardware could offer valuable insights, especially for constrained environments or dedicated cryptographic hardware.

9 Conclusion

Even though post-quantum schemes are theoretically secure, it is evident that the practical implementations introduce numerous attack vectors. This SoK provided a comprehensive analysis of SCA and their mitigations in both software and hardware implementations. By systematically breaking down attack strategies and examining their applicability across schemes, our goal was to support researchers and practitioners in understanding and mitigating real-world SCA risks in PQC deployments. A key insight from our analysis is the existence of shared vulnerabilities across different schemes and use cases, which allows certain attack strategies to generalize across cryptographic families. Our proposed evaluation guidelines show how existing SCA research can be systematically used to assess PQC implementations. Finally, we highlighted several open areas that warrant further investigation.

References

- [AAC⁺23] Gorjan Alagic, D Apon, D Cooper, Q Dang, T Dang, J Kelsey, J Lichtinger, C Miller, D Moody, R Peralta, et al. Status report on the fourth round of the nist post-quantum cryptography standardization process, 2023.
- [ABC⁺23] Melissa Azouaoui, Olivier Bronchain, Gaëtan Cassiers, Clément Hoffmann, Yulia Kuzovkova, Joost Renes, Tobias Schneider, Markus Schöner, François-Xavier Standaert, and Christine van Vredendaal. Protecting

dilithium against leakage: Revisited sensitivity analysis and improved implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):58–79, Aug. 2023.

- [ABH⁺22] Melissa Azouaoui, Olivier Bronchain, Clément Hoffmann, Yulia Kuzovkova, Tobias Schneider, and François-Xavier Standaert. Systematic study of decryption and re-encryption leakage: The case of kyber. In Josep Balasch and Colin O’Flynn, editors, *Constructive Side-Channel Analysis and Secure Design*, pages 236–256, Cham, 2022. Springer International Publishing.
- [ACD⁺22] Gorjan Alagic, David Cooper, Quynh Dang, Thinh Dang, John M. Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl A. Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Daniel Smith-Tone, and Daniel Apon. Status report on the third round of the nist post-quantum cryptography standardization process, 2022-07-05 04:07:00 2022.
- [ACK24] Thomas Aulbach, Fabio Campos, and Juliane Krämer. SoK: On the physical security of UOV-based signature schemes. *Cryptology ePrint Archive*, Paper 2024/1818, 2024.
- [ADP18] Martin R. Albrecht, Amit Deo, and Kenneth G. Paterson. Cold boot attacks on ring and module LWE keys under the NTT. *Cryptology ePrint Archive*, Paper 2018/672, 2018.
- [ATT⁺18] Aydin Aysu, Youssef Tobah, Mohit Tiwari, Andreas Gerstlauer, and Michael Orshansky. Horizontal side-channel vulnerabilities of post-quantum key exchange protocols. In *2018 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 81–88, 2018.
- [BBB⁺24] Estuardo Alpirez Bock, Gustavo Banegas, Chris Brzuska, Łukasz Chmielewski, Kirthivaasan Puniamurthy, and Milan Šorf. Breaking dpa-protected kyber via the pair-pointwise multiplication. In Christina Pöpper and Lejla Batina, editors, *Applied Cryptography and Network Security*, pages 101–130, Cham, 2024. Springer Nature Switzerland.
- [BBB⁺25] Daniel J. Bernstein, Karthikeyan Bhargavan, Shivam Bhasin, Anupam Chattopadhyay, Tee Kiah Chia, Matthias J. Kannwischer, Franziskus Kiefer, Thales B. Paiva, Prasanna Ravi, and Goutam Tamvada. Kyberslash: Exploiting secret-dependent division timings in kyber implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):209–234, Mar. 2025.
- [BBE⁺18] Gilles Barthe, Sonia Belaïd, Thomas Espitau, Pierre-Alain Fouque, Benjamin Grégoire, Mélissa Rossi, and Mehdi Tibouchi. Masking the glp lattice-based signature scheme at any order. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 354–384, Cham, 2018. Springer International Publishing.
- [BCM⁺23] Marcus Brinkmann, Chitchanok Chuengsatiansup, Alexander May, Julian Nowakowski, and Yuval Yarom. Leaky mceliece: Secret key recovery from highly erroneous side-channel information. *IACR Cryptol. ePrint Arch.*, page 1536, 2023.
- [BCO04a] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In Marc Joye and Jean-Jacques Quisquater, editors, *Cryptographic Hardware and Embedded Systems - CHES 2004*:

6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings, volume 3156 of *Lecture Notes in Computer Science*, pages 16–29. Springer, 2004.

- [BCO04b] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In Marc Joye and Jean-Jacques Quisquater, editors, *Cryptographic Hardware and Embedded Systems - CHES 2004*, pages 16–29, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
- [BCP⁺14] Lejla Batina, Łukasz Chmielewski, Louiza Papachristodoulou, Peter Schwabe, and Michael Tunstall. Online template attacks. In Willi Meier and Debdeep Mukhopadhyay, editors, *Progress in Cryptology – INDOCRYPT 2014*, volume 21–36 of *LNCS*, page 8885. SV, 2014. <http://cryptojedi.org/papers/#ota>.
- [BCP⁺17] Lejla Batina, Łukasz Chmielewski, Louiza Papachristodoulou, Peter Schwabe, and Michael Tunstall. Online template attacks. *Journal of Cryptographic Engineering*, pages 1–16, 2017. <http://cryptojedi.org/papers/#ota>, see also short version [BCP⁺14].
- [BDH⁺21] Shivam Bhasin, Jan-Pieter D’Anvers, Daniel Heinz, Thomas Pöppelmann, and Michiel Van Beirendonck. Attacking and defending masked polynomial comparison for lattice-based cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(3):334–359, Jul. 2021.
- [BDK⁺18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [BDK⁺21] Michiel Van Beirendonck, Jan-Pieter D’Anvers, Angshuman Karmakar, Josep Balasch, and Ingrid Verbauwhede. A side-channel-resistant implementation of SABER. *ACM J. Emerg. Technol. Comput. Syst.*, 17(2):10:1–10:26, 2021.
- [Ber09] Daniel J Bernstein. Introduction to post-quantum cryptography. In *Post-quantum cryptography*, pages 1–14. Springer, 2009.
- [BFM⁺19] Joppe W. Bos, Simon Friedberger, Marco Martinoli, Elisabeth Oswald, and Martijn Stam. Assessing the feasibility of single trace power analysis of frodo. In Carlos Cid and Michael J. Jacobson Jr., editors, *Selected Areas in Cryptography – SAC 2018*, pages 216–234, Cham, 2019. Springer International Publishing.
- [BGR⁺21] Joppe W Bos, Marc Gourjon, Joost Renes, Tobias Schneider, and Christine Van Vredendaal. Masking kyber: First-and higher-order implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 173–214, 2021.
- [BHK⁺19] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS ’19*, page 2129–2146, New York, NY, USA, 2019. Association for Computing Machinery.

- [BMG⁺24] Chloé Bâisse, Antoine Moran, Guillaume Goy, Julien Maillard, Nicolas Aragon, Philippe Gaborit, Maxime Lecomte, and Antoine Loiseau. Secret and shared keys recovery on hamming quasi-cyclic with SASCA. *Cryptology ePrint Archive*, Paper 2024/440, 2024.
- [BNGD23] Linus Backlund, Kalle Ngo, Joel Gärtner, and Elena Dubrova. Secret key recovery attack on masked and shuffled implementations of crystals-kyber and saber. In *International Conference on Applied Cryptography and Network Security*, pages 159–177. Springer, 2023.
- [BPO⁺20] Florian Bache, Clara Paglialonga, Tobias Oder, Tobias Schneider, and Tim Güneysu. High-speed masking for polynomial comparison in lattice-based kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(3):483–507, 2020.
- [BUC19] Utsav Banerjee, Tenzin S. Ukyab, and Anantha P. Chandrakasan. Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(4):17–61, 2019.
- [BVC⁺23] Alexandre Berzati, Andersson Calle Viera, Maya Chartouny, Steven Madec, Damien Vergnaud, and David Vigilant. Exploiting intermediate value leakage in dilithium: A template-based approach. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(4):188–210, 2023.
- [CCD⁺22] Po-Jen Chen, Tung Chou, Sanjay Deshpande, Norman Lahr, Ruben Niederhagen, Jakub Szefer, and Wen Wang. Complete and improved fpga implementation of classic mceliece. *Cryptology ePrint Archive*, 2022.
- [CDCG22] Brice Colombier, Vlad-Florin Dragoi, Pierre-Louis Cayrel, and Vincent Grosso. Message-recovery profiled side-channel attack on the classic mceliece cryptosystem. *IACR Cryptol. ePrint Arch.*, page 125, 2022.
- [CEvMS16] Cong Chen, Thomas Eisenbarth, Ingo von Maurich, and Rainer Steinwandt. Masking large keys in hardware: A masked implementation of mceliece. In Orr Dunkelman and Liam Keliher, editors, *Selected Areas in Cryptography – SAC 2015*, pages 293–309, Cham, 2016. Springer International Publishing.
- [CGTZ23] Jean-Sébastien Coron, François Gérard, Matthias Trannoy, and Rina Zeitoun. Improved gadgets for the high-order masking of dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):110–145, Aug. 2023.
- [CK13] Omar Choudary and Markus G. Kuhn. Efficient template attacks. *IACR Cryptol. ePrint Arch.*, page 770, 2013.
- [CKA⁺21] Zhaohui Chen, Emre Karabulut, Aydin Aysu, Yuan Ma, and Jiwu Jing. An efficient non-profiled side-channel attack on the crystals-dilithium post-quantum signature. In *2021 IEEE 39th International Conference on Computer Design (ICCD)*, pages 583–590, 2021.
- [CKKA23] Alvaro Cintas Canto, Jasmin Kaur, Mehran Mozaffari Kermani, and Reza Azarderakhsh. Algorithmic security is insufficient: A comprehensive survey on implementation attacks haunting post-quantum security. *arXiv preprint arXiv:2305.13544*, 2023.
- [CMJ23] Zhaohui Chen, Yuan Ma, and Jiwu Jing. Low-cost shuffling countermeasures against side-channel attacks for ntt-based post-quantum cryptography. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(1):322–326, 2023.

- [CRR03] Suresh Chari, Josyula R. Rao, and Pankaj Rohatgi. Template attacks. In Burton S. Kaliski, Çetin K. Koç, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2002*, pages 13–28, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [DG24] Haiyue Dong and Qian Guo. Ot-pca: New key-recovery plaintext-checking oracle based side-channel attacks on hqc with offline templates. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):251–274, Dec. 2024.
- [DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.
- [DKL⁺24] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Gregor Seiler, Peter Schwabe, and Damien Stehlé. PQ-CRYSTALS, Dilithium. <https://github.com/pq-crystals/dilithium>, 2024. GitHub repository.
- [DNGW23] Elena Dubrova, Kalle Ngo, Joel Gärtner, and Ruize Wang. Breaking a fifth-order masked implementation of crystals-kyber by copy-paste. In *Proceedings of the 10th ACM Asia Public-Key Cryptography Workshop*, pages 10–20, 2023.
- [dPKPR24] Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Racoon: A masking-friendly signature proven in the probing model. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, pages 409–444, Cham, 2024. Springer Nature Switzerland.
- [DR24] Loïc Demange and Mélissa Rossi. A provably masked implementation of BIKE key encapsulation mechanism. *Cryptology ePrint Archive*, Paper 2024/076, 2024.
- [DSDGR20] Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. Lwe with side information: Attacks and concrete security estimation. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, pages 329–358, Cham, 2020. Springer International Publishing.
- [DSGHK23] Dana Dachman-Soled, Huijing Gong, Tom Hanson, and Hunter Kippen. Revisiting security estimation for lwe with hints from a geometric perspective. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 748–781, Cham, 2023. Springer Nature Switzerland.
- [DXN⁺23] Sanjay Deshpande, Chuanqi Xu, Mamuri Nawan, Kashif Nawaz, and Jakub Szefer. Fast and efficient hardware implementation of hqc. In *International Conference on Selected Areas in Cryptography*, pages 297–321. Springer, 2023.
- [FHK⁺18] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-fourier lattice-based compact signatures over ntru. *Submission to the NIST’s post-quantum cryptography standardization process*, 36(5):1–75, 2018.

- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael Wiener, editor, *Advances in Cryptology — CRYPTO’ 99*, pages 537–554, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [FVBBR⁺21] Tim Fritzmann, Michiel Van Beirendonck, Debapriya Basu Roy, Patrick Karl, Thomas Schamberger, Ingrid Verbauwhede, and Georg Sigl. Masked accelerators and instruction set extensions for post-quantum cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):414–460, Nov. 2021.
- [GHJ⁺22] Qian Guo, Clemens Hlauschek, Thomas Johansson, Norman Lahr, Alexander Nilsson, and Robin Leander Schröder. Don’t reject this: Key-recovery timing attacks due to rejection-sampling in hqc and bike. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(3):223–263, Jun. 2022.
- [GJCJ23] Naina Gupta, Arpan Jati, Anupam Chattopadhyay, and Gautam Jha. Lightweight hardware accelerator for post-quantum digital signature crystals-dilithium. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2023.
- [GJJ22] Qian Guo, Andreas Johansson, and Thomas Johansson. A key-recovery side-channel attack on classic mceliece implementations. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):800–827, 2022.
- [GJN20] Qian Guo, Thomas Johansson, and Alexander Nilsson. A key-recovery timing attack on post-quantum primitives using the fujisaki-okamoto transformation and its application on frodokem. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, pages 359–386, Cham, 2020. Springer International Publishing.
- [GLG22] Guillaume Goy, Antoine Loiseau, and Philippe Gaborit. A new key recovery side-channel attack on hqc with chosen ciphertext. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography*, pages 353–371, Cham, 2022. Springer International Publishing.
- [GMGL24] Guillaume Goy, Julien Maillard, Philippe Gaborit, and Antoine Loiseau. Single trace hqc shared key recovery with sasca. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):64–87, March 2024.
- [GMRR22] Morgane Guerreau, Ange Martinelli, Thomas Ricosset, and Mélissa Rossi. The hidden parallelepiped is back again: Power analysis attacks on falcon. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(3):141–164, Jun. 2022.
- [GNNJ23] Qian Guo, Denis Nabokov, Alexander Nilsson, and Thomas Johansson. Sca-ldpc: A code-based framework for key-recovery side-channel attacks on post-quantum encryption schemes. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 203–236, Singapore, 2023. Springer Nature Singapore.
- [Gou01] Louis Goubin. A sound method for switching between boolean and arithmetic masking. In Çetin K. Koç, David Naccache, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems — CHES 2001*, pages 3–15, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.

- [HHP⁺21] Mike Hamburg, Julius Hermelink, Robert Primas, Simona Samardjiska, Thomas Schamberger, Silvan Streit, Emanuele Strieder, and Christine van Vredendaal. Chosen ciphertext k-trace attacks on masked cca2 secure kyber. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(4):88–113, Aug. 2021.
- [HKL⁺22] Daniel Heinz, Matthias J. Kannwischer, Georg Land, Thomas Pöppelmann, Peter Schwabe, and Amber Sprenkels. First-order masked kyber on ARM cortex-m4. *IACR Cryptol. ePrint Arch.*, page 58, 2022.
- [HLL⁺24] Tianze Huang, Jiahao Lu, Dongsheng Liu, Aobo Li, Shuo Yang, Lei Chen, and Xiang Li. A timing attack resistant lightweight post-quantum crypto-processor for sphincs+. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2024.
- [HLM⁺23] Clément Hoffmann, Benoît Libert, Charles Momin, Thomas Peters, and François-Xavier Standaert. Polka: Towards leakage-resistant post-quantum cca-secure public key encryption. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *Public-Key Cryptography – PKC 2023*, pages 114–144, Cham, 2023. Springer Nature Switzerland.
- [HPA21] James Howe, Thomas Prest, and Daniel Apon. Sok: How (not) to design and implement post-quantum cryptography. In *Cryptographers’ Track at the RSA Conference*, pages 444–477. Springer, 2021.
- [HPR⁺22] Anna-Lena Horlemann, Sven Puchinger, Julian Renner, Thomas Schamberger, and Antonia Wachter-Zeh. Information-set decoding with hints. In Antonia Wachter-Zeh, Hannes Bartz, and Gianluigi Liva, editors, *Code-Based Cryptography*, pages 60–83, Cham, 2022. Springer International Publishing.
- [HSMP24] Julius Hermelink, Silvan Streit, Erik Mårtensson, and Richard Petri. A generic framework for side-channel attacks against LWE-based cryptosystems. *Cryptology ePrint Archive*, Paper 2024/1211, 2024.
- [HZ12] Annelie Heuser and Michael Zohner. Intelligent machine homicide - breaking cryptographic devices using support vector machines. In Werner Schindler and Sorin A. Huss, editors, *Constructive Side-Channel Analysis and Secure Design - Third International Workshop, COSADE 2012, Darmstadt, Germany, May 3-4, 2012. Proceedings*, volume 7275 of *Lecture Notes in Computer Science*, pages 249–264. Springer, 2012.
- [JGCS24] Arpan Jati, Naina Gupta, Anupam Chattopadhyay, and Somitra Kumar Sanadhya. A configurable crystals-kyber hardware implementation with side-channel protection. *ACM Trans. Embed. Comput. Syst.*, 23(2), March 2024.
- [JMS⁺22] Petr Jedlicka, Lukas Malina, Petr Socha, Tomas Gerlich, Zdenek Martinasek, and Jan Hajny. On secure and side-channel resistant hardware implementations of post-quantum cryptography. In *Proceedings of the 17th International Conference on Availability, Reliability and Security, ARES ’22*, New York, NY, USA, 2022. Association for Computing Machinery.
- [JWN⁺23] Yanning Ji, Ruize Wang, Kalle Ngo, Elena Dubrova, and Linus Backlund. A side-channel attack on a hardware implementation of crystals-kyber. In *2023 IEEE European Test Symposium (ETS)*, pages 1–5, 2023.

- [KA21] Emre Karabulut and Aydin Aysu. Falcon down: Breaking falcon post-quantum signature scheme through side-channel attacks. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pages 691–696, 2021.
- [KA24] Emre Karabulut and Aydin Aysu. Masking falcon’s floating-point multiplication in hardware. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(4):483–508, Sep. 2024.
- [KA25] Merve Karabulut and Reza Azarderakhsh. Efficient CPA attack on hardware implementation of ML-DSA in post-quantum root of trust. Cryptology ePrint Archive, Paper 2025/009, 2025.
- [KdG21] Alexandre Karlov and Natacha Linard de Guertechin. Power analysis attack on kyber. Cryptology ePrint Archive, Paper 2021/1311, 2021.
- [KGB⁺18] Matthias J. Kannwischer, Aymeric Genêt, Denis Butin, Juliane Krämer, and Johannes Buchmann. Differential power analysis of XMSS and SPHINCS. In Junfeng Fan and Benedikt Gierlichs, editors, *Constructive Side-Channel Analysis and Secure Design - 9th International Workshop, COSADE 2018, Singapore, April 23-24, 2018, Proceedings*, volume 10815 of *Lecture Notes in Computer Science*, pages 168–188. Springer, 2018.
- [KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19*, pages 388–397. Springer, 1999.
- [KLH⁺20] Il-Ju Kim, Tae-Ho Lee, Jaeseung Han, Bo-Yeon Sim, and Dong-Guk Han. Novel single-trace ML profiling attacks on NIST 3 round candidate dilithium. Cryptology ePrint Archive, Paper 2020/1383, 2020.
- [KNAH22] Tendayi Kamucheka, Alexander Nelson, David Andrews, and Miaoqing Huang. A masked pure-hardware implementation of kyber cryptographic algorithm. Cryptology ePrint Archive, Paper 2022/1547, 2022.
- [Koc96] Paul C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In Neal Koblitz, editor, *Advances in Cryptology - CRYPTO ’96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.
- [KPR⁺] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-quantum crypto library for the ARM Cortex-M4. <https://github.com/mupq/pqm4>.
- [LBM15] Liran Lerman, Gianluca Bontempi, and Olivier Markowitch. A machine learning approach against a masked AES - reaching the limit of side-channel attacks with a learning model. *J. Cryptogr. Eng.*, 5(2):123–139, 2015.
- [LNPS20] Norman Lahr, Ruben Niederhagen, Richard Petri, and Simona Samardjiska. Side channel information set decoding using iterative chunking. In Shihao Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, pages 881–910, Cham, 2020. Springer International Publishing.
- [LVC24] Jonathan López-Valdivieso and René Cumplido. Design and implementation of hardware-software architecture based on hashes for sphincs+. *ACM Transactions on Reconfigurable Technology and Systems*, 2024.

- [MAB⁺18] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaleb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming quasi-cyclic (hqc). *NIST PQC Round*, 2(4):13, 2018.
- [MBBB16] Rauf Mahmudlu, Valentina Banciu, Lejla Batina, and Ileana Buhan. Lda-based clustering as a side-channel distinguisher. In Gerhard P. Hancke and Konstantinos Markantonakis, editors, *Radio Frequency Identification and IoT Security - 12th International Workshop, RFIDSec 2016, Hong Kong, China, November 30 - December 2, 2016, Revised Selected Papers*, volume 10155 of *Lecture Notes in Computer Science*, pages 62–75. Springer, 2016.
- [MGTF19] Vincent Migliore, Benoît Gérard, Mehdi Tibouchi, and Pierre-Alain Fouque. Masking dilithium. In Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung, editors, *Applied Cryptography and Network Security*, pages 344–362, Cham, 2019. Springer International Publishing.
- [MJB⁺24] Michail Moraitis, Yanning Ji, Martin Brisfors, Elena Dubrova, Niklas Lindskog, and Håkan Englund. Securing crystals-kyber in fpga using duplication and clock randomization. *IEEE Design & Test*, 41(5):7–16, 2024.
- [MN23] Alexander May and Julian Nowakowski. Too many hints – when lll breaks lwe. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 106–137, Singapore, 2023. Springer Nature Singapore.
- [MPP16] Houssein Maghrebi, Thibault Portigliatti, and Emmanuel Prouff. Breaking cryptographic implementations using deep learning techniques. In Claude Carlet, M. Anwar Hasan, and Vishal Saraswat, editors, *Security, Privacy, and Applied Cryptography Engineering - 6th International Conference, SPACE 2016, Hyderabad, India, December 14-18, 2016, Proceedings*, volume 10076 of *Lecture Notes in Computer Science*, pages 3–26. Springer, 2016.
- [MUTS22] Soundes Marzougui, Vincent Ulitzsch, Mehdi Tibouchi, and Jean-Pierre Seifert. Profiling side-channel attacks on dilithium: A small bit-fiddling leak breaks it all. *Cryptology ePrint Archive*, Paper 2022/106, 2022.
- [MWK⁺24] Catinca Muijdei, Lennert Wouters, Angshuman Karmakar, Arthur Beckers, Jose Maria Bermudo Mera, and Ingrid Verbauwhede. Side-channel analysis of lattice-based post-quantum cryptography: Exploiting polynomial multiplication. *ACM Transactions on Embedded Computing Systems*, 23(2):1–23, 2024.
- [Nat21] National Institute of Standards and Technology (NIST). Post-quantum cryptography standardization - round 3 submissions. <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>, 2021.
- [NDGJ21] Kalle Ngo, Elena Dubrova, Qian Guo, and Thomas Johansson. A side-channel attack on a masked IND-CCA secure saber KEM implementation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(4):676–707, 2021.
- [NDJ21] Kalle Ngo, Elena Dubrova, and Thomas Johansson. Breaking masked and shuffled CCA secure saber KEM by power analysis. In Chip-Hong Chang, Ulrich Rührmair, Stefan Katzenbeisser, and Debdeep Mukhopadhyay,

- editors, *ASHES@CCS 2021: Proceedings of the 5th Workshop on Attacks and Solutions in Hardware Security, Virtual Event, Republic of Korea, 19 November 2021*, pages 51–61. ACM, 2021.
- [NDKV24] Quinten Norga, Jan-Pieter D’Anvers, Suparna Kundu, and Ingrid Verbauwhede. X2X: Low-randomness and high-throughput A2B and B2A conversions for $d + 1$ shares in hardware. *Cryptology ePrint Archive*, Paper 2024/114, 2024.
- [NIS01] NIST. Security requirements for cryptographic modules. <https://doi.org/10.6028/NIST.FIPS.140-2>, 2001.
- [NIS17] NIST. Post-quantum cryptography standardization. <https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization>, 2017.
- [NIS22] NIST. Request for additional post-quantum cryptography digital signature schemes. <https://csrc.nist.gov/projects/pqc-dig-sig/round-2-additional-signatures>, 2022.
- [NR06] Phong Q. Nguyen and Oded Regev. Learning a parallelepiped: Cryptanalysis of ggh and ntru signatures. In Serge Vaudenay, editor, *Advances in Cryptology - EUROCRYPT 2006*, pages 271–288, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [ÖOP03] Siddika Berna Örs, Elisabeth Oswald, and Bart Preneel. Power-analysis attacks on an fpga – first experimental results. In Colin D. Walter, Çetin K. Koç, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2003*, pages 35–50, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [OSPG16] Tobias Oder, Tobias Schneider, Thomas Pöppelmann, and Tim Güneysu. Practical CCA2-secure and masked ring-LWE implementation. *Cryptology ePrint Archive*, Paper 2016/1109, 2016.
- [OST06] Dag Arne Osvik, Adi Shamir, and Eran Tromer. Cache attacks and countermeasures: The case of AES. In David Pointcheval, editor, *Topics in Cryptology - CT-RSA 2006, The Cryptographers’ Track at the RSA Conference 2006, San Jose, CA, USA, February 13-17, 2006, Proceedings*, volume 3860 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2006.
- [PHJ⁺17] Stjepan Picek, Annelie Heuser, Alan Jovic, Simone A. Ludwig, Sylvain Guilley, Domagoj Jakobovic, and Nele Mentens. Side-channel analysis and machine learning: A practical perspective. In *2017 International Joint Conference on Neural Networks, IJCNN 2017, Anchorage, AK, USA, May 14-19, 2017*, pages 4095–4102. IEEE, 2017.
- [PP19] Peter Pessl and Robert Primas. More practical single-trace attacks on the number theoretic transform. In Peter Schwabe and Nicolas Thériault, editors, *Progress in Cryptology – LATINCRYPT 2019*, pages 130–149, Cham, 2019. Springer International Publishing.
- [PPM17] Robert Primas, Peter Pessl, and Stefan Mangard. Single-trace side-channel attacks on masked lattice-based encryption. In Wieland Fischer and Naofumi Homma, editors, *Cryptographic Hardware and Embedded Systems – CHES 2017*, pages 513–533, Cham, 2017. Springer International Publishing.

- [PT20] Thales Bandiera Paiva and Routo Terada. A timing attack on the hqc encryption scheme. In Kenneth G. Paterson and Douglas Stebila, editors, *Selected Areas in Cryptography – SAC 2019*, pages 551–573, Cham, 2020. Springer International Publishing.
- [QLZ⁺23] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Jingdian Ming, Chengbin Jin, and Huizhong Li. Practical public template attacks on crystals-dilithium with randomness leakages. *IEEE Transactions on Information Forensics and Security*, 18:1–14, 2023.
- [RBMG21] Jan Richter-Brockmann, Johannes Mono, and Tim Güneysu. Folding bike: Scalable hardware implementation for reconfigurable devices. *IEEE Transactions on Computers*, 71(5):1204–1215, 2021.
- [RBRC22] Prasanna Ravi, Shivam Bhasin, Sujoy Sinha Roy, and Anupam Chattopadhyay. On exploiting message leakage in (few) nist pqc candidates for practical message recovery attacks. *IEEE Transactions on Information Forensics and Security*, 17:684–699, 2022.
- [RBVB23] Rafael Carrera Rodriguez, Florent Bruguier, Emanuele Valea, and Pascal Benoit. Correlation electromagnetic analysis on an fpga implementation of crystals-kyber. In *2023 18th Conference on Ph. D Research in Microelectronics and Electronics (PRIME)*, pages 217–220. IEEE, 2023.
- [RCDB24] Prasanna Ravi, Anupam Chattopadhyay, Jan Pieter D’Anvers, and Anubhab Baksi. Side-channel and fault-injection attacks over lattice-based post-quantum schemes (kyber, dilithium): Survey and new results. *ACM Trans. Embed. Comput. Syst.*, 23(2), March 2024.
- [RJH⁺18] Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. Side-channel assisted existential forgery attack on dilithium - a NIST PQC candidate. Cryptology ePrint Archive, Paper 2018/821, 2018.
- [RPBC20] Prasanna Ravi, Romain Poussier, Shivam Bhasin, and Anupam Chattopadhyay. On configurable sca countermeasures against single trace attacks for the ntt. In Lejla Batina, Stjepan Picek, and Mainack Mondal, editors, *Security, Privacy, and Applied Cryptography Engineering*, pages 123–146, Cham, 2020. Springer International Publishing.
- [RRCB20] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on cca-secure lattice-based pke and kems. *IACR transactions on cryptographic hardware and embedded systems*, pages 307–335, 2020.
- [RRCC24] Kamal Raj, Prasanna Ravi, Tee Kiah Chia, and Anupam Chattopadhyay. Improved ML-DSA hardware implementation with first order masking countermeasure. Cryptology ePrint Archive, Paper 2024/1817, 2024.
- [RRD⁺23] Gokulnath Rajendran, Prasanna Ravi, Jan-Pieter D’anvers, Shivam Bhasin, and Anupam Chattopadhyay. Pushing the limits of generic side-channel attacks on lwe-based kems - parallel pc oracle attacks on kyber kem and beyond. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023.

- [RSRVV15] Oscar Reparaz, Sujoy Sinha Roy, Frederik Vercauteren, and Ingrid Verbauwhede. A masked ring-lwe implementation. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 683–702. Springer, 2015.
- [SAW⁺23] Michael Schmid, Dorian Amiet, Jan Wendler, Paul Zbinden, and Tao Wei. Falcon takes off-a hardware implementation of the falcon signature scheme. *Cryptology ePrint Archive*, 2023.
- [Sen21] Nicolas Sendrier. Secure sampling of constant-weight words – application to BIKE. *Cryptology ePrint Archive*, Paper 2021/1631, 2021.
- [SFG24] Douglas Stebila, Scott Fluhrer, and Shay Gueron. Hybrid key exchange in TLS 1.3. Internet-Draft draft-ietf-tls-hybrid-design-11, Internet Engineering Task Force, October 2024. Work in Progress.
- [SFW24] Maximilian Schöffel, Johannes Feldmann, and Norbert Wehn. Efficient hardware implementation of constant time sampling for hqc, 2024.
- [SGG24] Robin Leander Schröder, Stefan Gast, and Qian Guo. Divide and surrender: Exploiting variable division instruction timing in HQC key recovery attacks. *Cryptology ePrint Archive*, Paper 2024/299, 2024.
- [SH24] Aein Rezaei Shahmirzadi and Michael Hutter. Efficient boolean-to-arithmetic mask conversion in hardware. *Cryptology ePrint Archive*, Paper 2024/1633, 2024.
- [SHR⁺22] Thomas Schamberger, Lukas Holzbaur, Julian Renner, Antonia Wachter-Zeh, and Georg Sigl. A power side-channel attack on the reed-muller reed-solomon version of the hqc cryptosystem. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography*, pages 327–352, Cham, 2022. Springer International Publishing.
- [SKL⁺20] Bo-Yeon Sim, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Tae-Ho Lee, Jaeseung Han, Hyojin Yoon, Jihoon Cho, and Dong-Guk Han. Single-trace attacks on message encoding in lattice-based kems. *IEEE Access*, 8:183175–183191, 2020.
- [SKS09] François-Xavier Standaert, François Koeune, and Werner Schindler. How to compare profiled side-channel attacks? In Michel Abdalla, David Pointcheval, Pierre-Alain Fouque, and Damien Vergnaud, editors, *Applied Cryptography and Network Security, 7th International Conference, ACNS 2009, Paris-Rocquencourt, France, June 2-5, 2009. Proceedings*, volume 5536 of *Lecture Notes in Computer Science*, pages 485–498, 2009.
- [SLKG23] Hauke Steffen, Georg Land, Lucie Kogelheide, and Tim Güneysu. Breaking and protecting the crystal: Side-channel analysis of dilithium in hardware. In Thomas Johansson and Daniel Smith-Tone, editors, *Post-Quantum Cryptography*, pages 688–711, Cham, 2023. Springer Nature Switzerland.
- [SÖP04] François-Xavier Standaert, Siddika Berna Örs, and Bart Preneel. Power analysis of an fpga. In Marc Joye and Jean-Jacques Quisquater, editors, *Cryptographic Hardware and Embedded Systems - CHES 2004*, pages 30–44, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
- [SPH22] Bo-Yeon Sim, Aesun Park, and Dong-Guk Han. Chosen-ciphertext clustering attack on crystals-kyber using the side-channel leakage of barrett reduction. *IEEE Internet of Things Journal*, 9(21):21382–21397, 2022.

- [SSMS10] Abdulhadi Shoufan, Falko Strenzke, H. Gregor Molter, and Marc Stöttinger. A timing attack against patterson algorithm in the mceliece pkc. In Donghoon Lee and Seokhie Hong, editors, *Information, Security and Cryptology – ICISC 2009*, pages 161–175, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [STM⁺08] Falko Strenzke, Erik Tews, H. Gregor Molter, Raphael Overbeck, and Abdulhadi Shoufan. Side channels in the mceliece pkc. In Johannes Buchmann and Jintai Ding, editors, *Post-Quantum Cryptography*, pages 216–229, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [SVP⁺24] Maxime Spyropoulos, David Vigilant, Fabrice Perion, Renaud Pacalet, and Laurent Sauvage. Masked vector sampling for HQC. *Cryptology ePrint Archive*, Paper 2024/1106, 2024.
- [SZN23] Ari Shaller, Linir Zamir, and Mehrdad Nojoumian. Roadmap of post-quantum cryptography standardization: Side-channel attacks and countermeasures. *Information and Computation*, 295:105112, 2023.
- [Tea] The PQClean Team. Pqclean: Post-quantum clean implementations - kyber512. https://github.com/PQClean/PQClean/blob/8db3ba/crypto_kem/kyber512/clean/.
- [TUX⁺23] Yutaro Tanaka, Rei Ueno, Keita Xagawa, Akira Ito, Junko Takahashi, and Naofumi Homma. Multiple-valued plaintext-checking side-channel attacks on post-quantum kems. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):473–503, Jun. 2023.
- [UXT⁺21] Rei Ueno, Keita Xagawa, Yutaro Tanaka, Akira Ito, Junko Takahashi, and Naofumi Homma. Curse of re-encryption: A generic power/em analysis on post-quantum kems. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):296–322, Nov. 2021.
- [VCGS14] Nicolas Veyrat-Charvillon, Benoît Gérard, and François-Xavier Standaert. Soft analytical side-channel attacks. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology – ASIACRYPT 2014*, pages 282–296, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.
- [WCCL22] Jian Wang, Weiqiong Cao, Hua Chen, and Haoyuan Li. Practical side-channel attack on masked message encoding in latticed-based KEM. *Cryptology ePrint Archive*, Paper 2022/859, 2022.
- [WCCL24] Jian Wang, Weiqiong Cao, Hua Chen, and Haoyuan Li. Blink: Breaking lattice-based schemes implemented in parallel with chosen-ciphertext attack. *Cryptology ePrint Archive*, Paper 2024/036, 2024.
- [WGL⁺24] Huaxin Wang, Yiwen Gao, Yuejun Liu, Qian Zhang, and Yongbin Zhou. In-depth correlation power analysis attacks on a hardware implementation of crystals-dilithium. *Cybersecurity*, 7(1):21, Jun 2024.
- [WHW⁺25] Congming Wei, Shulin He, An Wang, Shaofei Sun, Yaoling Ding, Jingqi Zhang, and Liehuang Zhu. An intelligent framework for cluster-based side-channel analysis on public-key cryptosystems. *IEEE Internet of Things Journal*, 12(2):1962–1973, 2025.

- [WND23] Ruize Wang, Kalle Ngo, and Elena Dubrova. A message recovery attack on lwe/lwr-based pke/kems using amplitude-modulated em emanations. In Seung-Hyun Seo and Hwajeong Seo, editors, *Information Security and Cryptology – ICISC 2022*, pages 450–471, Cham, 2023. Springer Nature Switzerland.
- [WNGD23] Ruize Wang, Kalle Ngo, Joel Gärtner, and Elena Dubrova. Single-trace side-channel attacks on CRYSTALS-dilithium: Myth or reality? Cryptology ePrint Archive, Paper 2023/1931, 2023.
- [WPH⁺22] Yingchen Wang, Riccardo Paccagnella, Elizabeth Tang He, Hovav Shacham, Christopher W. Fletcher, and David Kohlbrenner. Hertzbleed: Turning power Side-Channel attacks into remote timing attacks on x86. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 679–697, Boston, MA, August 2022. USENIX Association.
- [WPW⁺23] Yingchen Wang, Riccardo Paccagnella, Alan Wandke, Zhao Gang, Grant Garrett-Grossman, Christopher W. Fletcher, David Kohlbrenner, and Hovav Shacham. Dvfs frequently leaks secrets: Hertzbleed attacks beyond sike, cryptography, and cpu-only data. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2306–2320, 2023.
- [WTBB⁺19] Guillaume Wafo-Tapa, Slim Bettaieb, Loic Bidoux, Philippe Gaborit, and Etienne Marcatel. A practicable timing attack against HQC and its countermeasure. Cryptology ePrint Archive, Paper 2019/909, 2019.
- [XIU⁺21] Keita Xagawa, Akira Ito, Rei Ueno, Junko Takahashi, and Naofumi Homma. Fault-injection attacks against nist’s post-quantum cryptography round 3 kem candidates. In *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part II 27*, pages 33–61. Springer, 2021.
- [XL21] Yufei Xing and Shuguo Li. A compact hardware implementation of cca-secure key exchange mechanism crystals-kyber on fpga. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(2):328–356, Feb. 2021.
- [XPR⁺22] Zhuang Xu, Owen Pemberton, Sujoy Sinha Roy, David Oswald, Wang Yao, and Zhiming Zheng. Magnifying side-channel leakage of lattice-based cryptosystems with chosen ciphertexts: The case study of kyber. *IEEE Transactions on Computers*, 71(9):2163–2176, 2022.
- [XWT24] Dejun Xu, Kai Wang, and Jing Tian. A hardware-friendly shuffling countermeasure against side-channel attacks for kyber, 2024.
- [YRZ⁺23] Bolin Yang, Prasanna Ravi, Fan Zhang, Ao Shen, and Shivam Bhasin. STAMP-single trace attack on m-LWE pointwise multiplication in kyber. Cryptology ePrint Archive, Paper 2023/1184, 2023.
- [ZBT19] Timo Zijlstra, Karim Bigou, and Arnaud Tisserand. Fpga implementation and comparison of protections against scas for rlwe. In *Progress in Cryptology–INDOCRYPT 2019: 20th International Conference on Cryptology in India, Hyderabad, India, December 15–18, 2019, Proceedings 20*, pages 535–555. Springer, 2019.

- [ZLYW23] Shiduo Zhang, Xiuhan Lin, Yang Yu, and Weijia Wang. Improved power analysis attacks on falcon. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 565–595, Cham, 2023. Springer Nature Switzerland.
- [ZPM⁺23] Yiqiang Zhao, Shijian Pan, Haocheng Ma, Ya Gao, Xintong Song, Jiaji He, and Yier Jin. Side channel security oriented evaluation and protection on hardware implementations of kyber. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 70(12):5025–5035, 2023.